

POLITECNICO
MILANO 1863

SCUOLA DI INGEGNERIA INDUSTRIALE
E DELL'INFORMAZIONE

Tesi di Laurea Magistrale in
Computer Science and Engineering
Ingegneria Informatica

Binary Instrumentation for Tracking Edge Coverage in Fuzzing using GRAPE binary rewriter

Author:
Scandroglio Edoardo

Student ID:
216600

Advisor:
Prof. Name Surname

Academic Year:
2026-27

Dedicated to someone.

Abstract

...

Keywords: key, words, go, here

Abstract in lingua italiana

...

Parole chiave: qui, vanno, le, parole, chiave

Contents

Abstract **i**

Abstract in lingua italiana **iii**

Contents **v**

1 Background and Motivation **1**
1.1 Fuzzing Background . . . 1
1.2 Binary Rewriting Background . . . 4
1.3 ELF Loading and Relocations . . . 7
1.3.1 ELF Structure . . . 7
1.3.2 ELF Linking . . . 8
1.3.3 ELF Loading . . . 9
1.3.4 ELF Symbols and Relocations . . . 12
1.4 Frame pointer omission and Red Zone . . . 14
1.5 Problem Statement . . . 16
1.6 State of the Art . . . 17
1.6.1 AFL Functioning . . . 19
1.7 Goals and Challenges . . . 22

2 Approach **25**
2.1 Forkserver Insertion . . . 25
2.2 Shared Memory Instrumentation . . . 28
2.3 Instrumentation Points Search . . . 30
2.4 Actual Instrumentation . . . 34
2.4.1 RFLAGS . . . 36
2.5 Forkserver Optimization . . . 37
2.6 Dummy Shared Memory . . . 38

3 Implementation **41**
3.1 Forkserver Specification 41
3.2 Instrumentator Specification 43
3.2.1 Binary Modification 44
3.2.2 Instrumentation 47

4 Experiments **55**
4.1 Goals 55
4.2 Datasets and Experiment Setup 56
4.3 Experiment I 57
4.4 Experiment II 58

5 Limitations and Future Works **63**
5.1 Limitations 63
5.2 Future Works 64

6 Conclusion **65**
6.1 Summary of the work 65
6.2 Assessment of the goals 66
6.3 Final remarks 67

Bibliography **69**

List of Acronyms **73**

List of Figures **75**

List of Tables **77**

List of Algorithms **79**

1 | Background and Motivation

This chapter will focus on presenting the context of this thesis. In particular, in Section 1.1 the fundamentals of fuzzing will be discussed, with particular attention to coverage-guided grey-box fuzzing. In Section 1.2 the background on binary rewriting is discussed, along with its approaches and difficulties. In the next chapther will be presented the approach used in order to resolve the main problems regarding the solution discussed in this thesis. In order to do so some concept of binary loading must be presented (Section 1.3). In order to better understand a motivation to a design choice expressed in section 2.4, in Section 1.4 are presented two of the compiler optimization in terms of function preamble. Section 1.5 states the problem addressed by this work, and Section 1.6 reviews the state of the art in binary-only fuzzing, also focusing on a description of AFL/AFL++ internals. In the last section (Section 1.7) I will present the main goals and challenges of this thesis, and explain why the implemented tool differs from the state of the art.

1.1. Fuzzing Background

Fuzzing is nowadays a widely used approach for software testing and vulnerability assessment [23, 35]. Its main strength derives from the fact that it can rapidly produce various inputs to be fed into a target executable. In general, a fuzzer can be divided into the **Input Generator**, which is responsible for providing inputs to the **Executor** that executes them. During the execution, the **Defect Monitor** checks whether the executed input causes a crash of the target.
In particular, the Input Generator must choose which input to provide to the executor among all the inputs in the *Input Space*. A subset of the *Input Space* is represented by the *Valid Input Space*. This is a subset of the input space that specifically contains all the inputs that are well formed with respect to the application being tested. For example, if the fuzzed target is a PDF reader, a valid input is a PDF file. A good Input Generator must choose inputs that are also valid inputs in order to increment the probability of finding deeper bugs [35]. Hence the need for a precise input selection and mutation strategy arises, in order to discover as many edge cases as possible.

In order to do so, the Input Generator must obtain some information from the Executor. Depending on the amount of information the Input Generator has, fuzzing can be classified into *White-box*, *Black-box* or *Grey-box*. In Black-box fuzzing, the fuzzer does not have any knowledge of the execution [25]. In White-box fuzzing, instead, the source code must be available in the general case, and techniques such as Symbolic and Concolic execution are used [16]. In Grey-box fuzzing, instead, the inputs are chosen using, for example, **Code Coverage** information (i.e. the number of lines of code visited during execution of the target). This approach is called *Coverage-Guided Grey-box fuzzing* and is based on prioritizing the inputs that discover new sections of code. Since the source code is not always available, Code Coverage is not always a viable Coverage Measure. In general, the coverage measures can be of different granularity [14]. The three main options are:

- **Block Coverage**: Keeps track of the hit count for each Basic Block (BB). This approach is used by some popular fuzzers such as libFuzzer or honggfuzz [14]. This method is often too imprecise because it fails to capture the transitions between BBs.
- **Edge Coverage**: Keeps track of the hit count of every edge in the Control Flow Graph (CFG) (i.e. the transition between two BBs). This is the approach used by AFL and AFL++ [12]. It maintains overheads similar to Block Coverage but can track the transitions between BBs and not only their execution, adding some context information. However, it lacks precision in keeping track of the order in which the edges are taken.
- **Path Coverage**: This is the most complete metric and keeps track of the complete path taken during the execution of a specific input, thus also keeping track of the order in which the edges are taken. The main drawback of this metric is that a single execution path could become very large, resulting in large runtime and memory overheads.

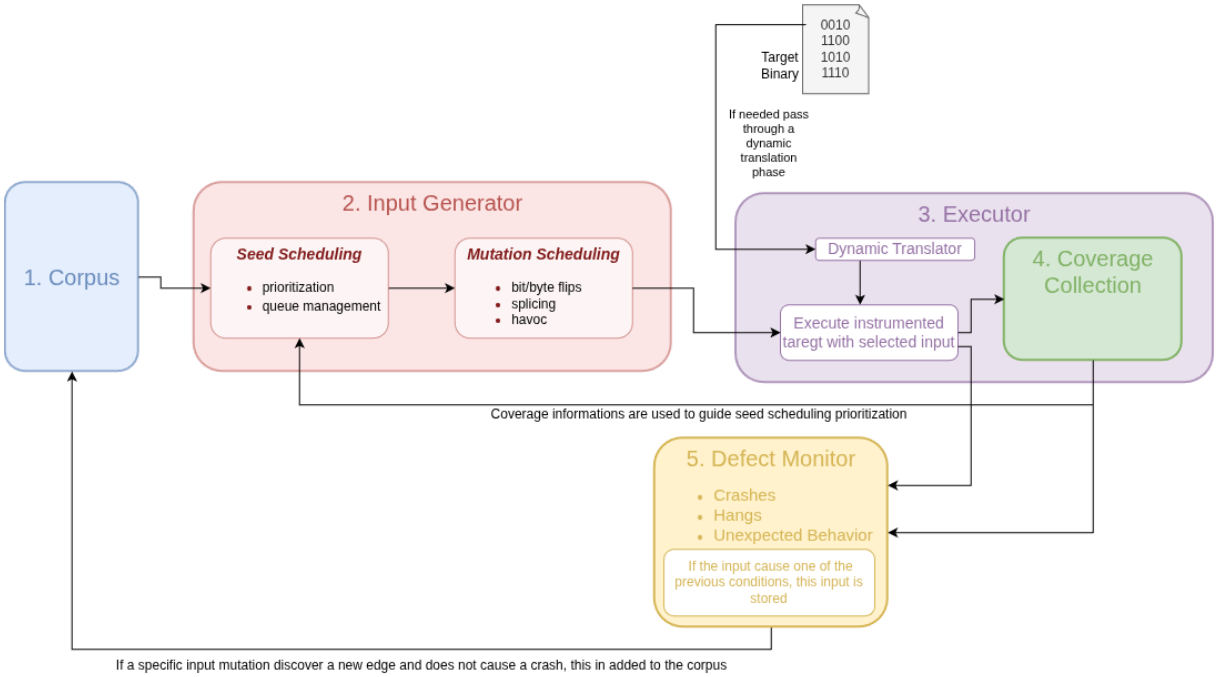
One of the best approach to Code Coverage, in terms of tradeoff between overhead and effectiveness, is **Edge Coverage** [14], which keeps track of which edges in the CFG are taken during execution without the overheads incurred by keeping track of the entire execution path. Coverage information is obtained through the instrumentation of the target application. Depending on the availability of the source code, the instrumentation can be added in different ways:

- The source code is available: In this case the instrumentation is added at *Compile Time*.

- The source code is not available: In this case there are two different techniques for instrumenting the application:
  - **Static Binary Instrumentation (SBI)**: The instrumentation is added before the target is executed, without access to the source code, relying on Static Binary Rewriting techniques.
  - **Dynamic Binary Instrumentation (DBI)**: The instrumentation is added during program execution, relying on Dynamic Binary Rewriting techniques.

Regardless of the availability of the source code, the *Coverage-Guided Grey-box fuzzing* technique can be summarized as shown in fig. 1.1.

Figure 1.1: Generic diagram describing the Coverage-Guided Grey-box fuzzing workflow



Specifically for **Edge Coverage**, fig. 1.1 can be described as follows:

1. **Seed Corpus**: Represents the set of inputs on which mutations are computed. At the beginning it is composed of user-defined seeds, but during fuzzing this set can grow if a specific mutation discovers new edges not yet found.
2. **Input Generator**: Composed of two main parts:
   - **Seed Scheduling**: responsible for deciding which seed in the corpus to mutate.

- **Mutation Scheduling**: responsible for deciding which mutation or mutations to apply to the selected seed.

3. **Executor**: The component that actually executes the target binary. If the target binary has already been instrumented with Edge Coverage tracking instructions, it is executed as is; otherwise, in the case of DBI, a dynamic translator must dynamically instrument the binary.

4. **Coverage Collection**: In the case of Edge Coverage, this part is represented by the data structure used to keep track of which edges between BBs have been taken and which have not.
   This information is used, in some tools such as AFL++ [12], to guide Seed Scheduling, a technique that, given different inputs with the same level of coverage (i.e. number of edges visited), prioritizes the input that is smaller in size and takes less execution time.

5. **Defect Monitor**: This component checks the exit status of the target to catch any crash or timeout of the target, or any other unexpected behavior. If none of these conditions are met, meaning that the target ends normally, the coverage is checked to see whether the current input discovered a new edge. If the input discovered a new edge, that input is added to the corpus.

1.2. Binary Rewriting Background

Binary rewriting consists in altering an already compiled executable without having access to the source code, while ensuring that the modified binary remains executable. The process of binary rewriting can be classified into Static Binary Rewriting, if the binary is stored in persistent memory while being rewritten, and Dynamic Binary Rewriting, if the binary is being executed. The process of binary rewriting can be divided into four main steps [31]:

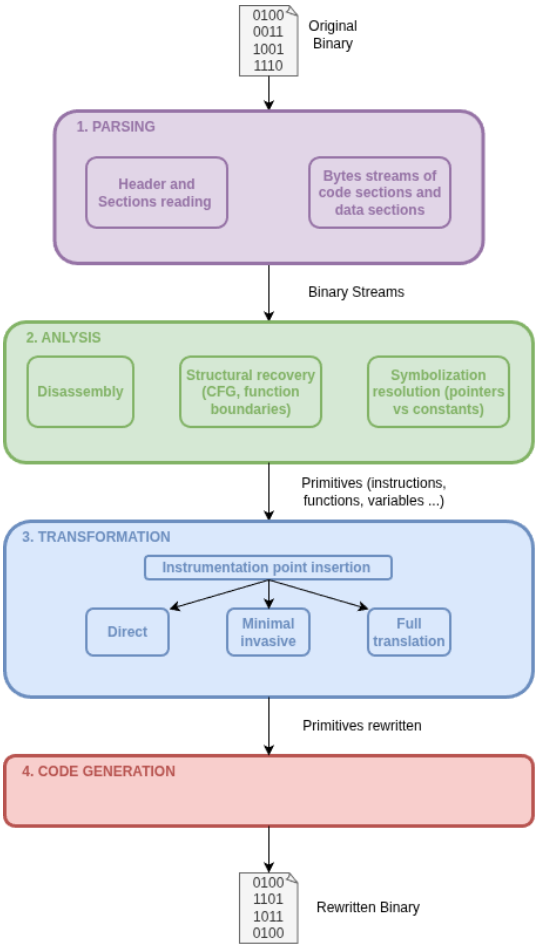
1. *Parsing*: In this phase the main goal is to retrieve information about the binary. In particular, the output of this phase is a raw stream of instructions (in machine code) and data. This information is then used in the Analysis step.

2. *Analysis*: The information gained from the Parsing phase is analyzed to reconstruct the program structure. For example, the raw stream of machine code is passed through a disassembler that reconstructs the assembly instructions. A structural recovery is performed, which includes function detection and CFG reconstruction, and labels and symbols are extracted from the raw stream of data.

3. *Transformation*: Once the CFG has been reconstructed and function and data boundaries have been recovered, the binary rewriter can modify the binary at the Instrumentation Point (IP). These are user-defined locations in the binary where the rewriter must add, modify or delete an instruction. The IP can be coarse- or fine-grained. In the first case, for example, the instrumentation could be inserted at branch level (at the beginning of each BB). In the second case, instead, any instruction could potentially be modified.

4. *Code Generation*: In this last step, the changes made in the previous steps must be integrated into the binary while preserving its executability. This can be done in different ways. For example, the changes could be inserted into a new section of the executable, redirecting execution to the new code at the IP. Alternatively, this step could also be performed by reassembling a completely new executable starting from the changes made at the IP. This second approach is much harder, because it requires preserving all the references inside the executable and preserving the executable layout.

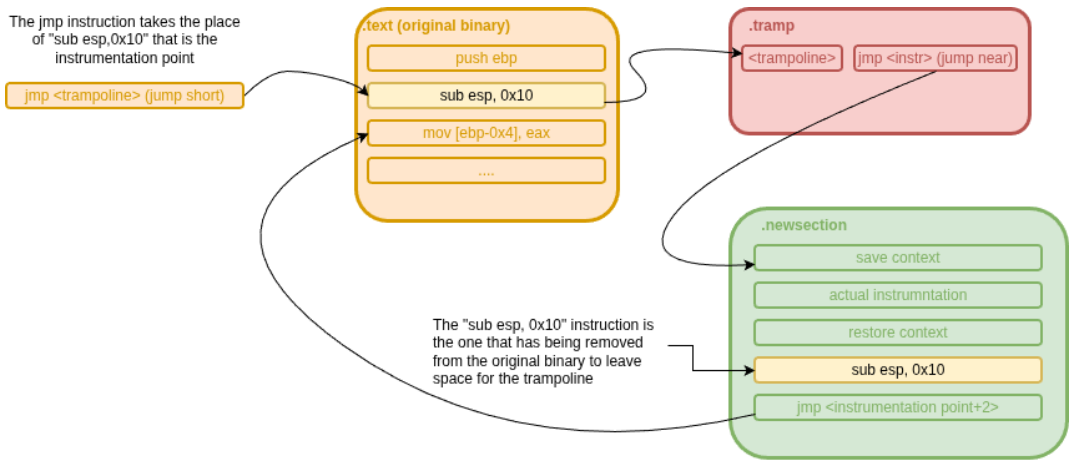
Figure 1.2: An example of a Static Binary Rewriting Workflow



In fig. 1.2, a general scheme for static binary rewriting can be seen. In this case all the phases occur prior to execution, so the cost in terms of overhead is paid only once, before the execution. The main problems are disambiguating between code and data (i.e. deciding whether a stream of bytes represents machine code instructions or constants placed inline in the .text section) and resolving indirect branches [3, 31]. This problem is generally undecidable and can only be solved using heuristics. There are three main families of static transformation:

- **Direct**: Consists in directly modifying the instruction. Works well with RISC systems with instructions having the same length. Generally fails when adding or removing instructions, because this does not preserve the offset into the .text section.
- **Minimal-invasive**: The new code is inserted into a new section of the binary, and at the IP an unconditional jump to the new section is placed. The main limitation of this approach arises when the new section is far from the IP and a near jump (necessary to reach the new section) uses a different number of bytes than the instruction at the instrumentation point. In fact, in x86-64 assembly a short jump uses 2 bytes while a near jump uses 5 bytes. So, if a short jump is needed at the instrumentation point for alignment reasons, but the new section is too far away and a near jump is needed instead, it is necessary to add a **trampoline**. This approach can be seen in fig. 1.3. Trampoline-based instrumentation is one of the most common approaches to binary rewriting [31].
- **Full-translation**: The binary is lifted into an Intermediate Representation (IR) that makes it possible to add an IP at instruction level. The IR is then reassembled into machine code after the changes are made.

Figure 1.3: An example of Static Instrumentation using the trampoline technique



With regard to Dynamic Binary Rewriting, the Analysis and Transformation phases are executed at runtime for each instruction. In this case, none of the problems that arise in Static Binary Rewriting occur, since the binary is being executed, so indirect branches can be resolved using the values in the registers, and there is no need to disambiguate between constants inside the .text section and instructions. The main drawback of this approach is the prohibitive overhead caused by the dynamic analysis and transformation of each instruction, together with the fact that only the taken branches can be modified. In fact, if a branch is not actually taken during execution, its instructions never reach the rewriter.

A specific type of binary rewriting is Binary Instrumentation. Binary instrumentation uses binary rewriting techniques to add instructions that track binary behaviour, log events, or, in the case of this thesis, insert edge coverage tracing instructions.

1.3. ELF Loading and Relocations

Executable and Linkable Format (ELF) is the standard executable format under Linux operating systems. The work done in this thesis heavily involves the ELF structure, with particular focus on some sections used for relocation, alongside the process of executing an ELF. For this reason the following section will disclose in details some of the ELF loading and relocation mechanism. But, in order to do so, a review of the ELF basic structure must be done.

1.3.1. ELF Structure

In this subsection the process of ELF execution is discussed, with a focus on the parts of the execution flow that will influence this work.
The **ELF Header** is a structure located at the beginning of the ELF and contains some important information about the type of ELF, the target operating system .... Alongside this information there is the location of two important data structures inside the ELF:

- The **Program Header Table**, which defines the *Execution View*
- The **Section Header Table**, which defines the *Linking View*

The **Program Header Table** contains information needed to create a runtime image of the program in memory, via the **Segments** *(i.e. which parts of the file must be loaded into memory, which shared objects are needed ...)*. Each segment contains the access flags for that part of memory (i.e. Read/Write/Execute). The **Section Header Table** contains information about the **Sections** that compose the binary, together with their

mapping to the segments (e.g. which segment the *.text* section is mapped to). Among all the sections, some contain specific structures used to perform specific tasks:

- ***.dynamic**: The dynamic section contains information used by the Dynamic Linker (e.g. the dependencies, the interpreter location . . .).*
- ***.dynsym**: List of symbols defined and used by the binary. If the binary is not stripped, there is also the **.symtab** section, which contains a more complete list of symbols.*
- ***.hash or .gnu.hash**: Structures used for efficient symbol lookup during symbol resolution*[1].
- ***.rela.dyn and .rela.plt**: Both are relocation tables. The first is the standard one, the second is used in the case of **Lazy Binding**.*
- ***.shstrtab, .dynstr and .strtab**: Hold lists of strings referenced by other parts of the ELF. The first holds section names, the second holds the names of the symbols contained in the **.dynsym** section, and the last one holds the names of the entire **.symtab** section.*
- ***.got and .got.plt**: Contain the Global Offset Table (GOT), filled by the dynamic linker with the addresses of external functions and variables. The first is the standard one, the second is used by the Procedure Linkage Table (PLT).*
- ***.plt**: Contains the PLT, which is an executable section made up of trampolines. In particular, it contains stubs for functions that might be located in another ELF or Shared Object (SO). The stub references the corresponding GOT entry containing the resolved address of the function. This section is used in the case of **Lazy Binding**.*
- ***.init and .init_array**: Sections used for initialization. **.init** contains initialization code, while **.init_array** contains the addresses of initialization functions.*
- ***.fini and .fini_array**: Sections used for termination handling. **.fini** contains termination code, while **.fini_array** contains the addresses of termination functions.*

1.3.2. ELF Linking

Before actually digging deep into ELF loading, the linking process must be reviewed. The **Linking phase** consists of joining together different pieces of code belonging to different files. For example, if a program uses the `printf` function, it needs to take the

[1] The **.gnu.hash** section is the more efficient version of **.hash**.

code of printf from the libc library. This process of joining together the executable that uses printf with the printf code located in another file is called **Linking**. Linking can be of two different types:

- **Static linking**: The needed code (e.g. the instructions of printf) is copied inside the binary at *Link Time*. This causes the size of the binary to be bigger (because the code of external functions actually lies inside it) and consequently wastes RAM space (if 10 binaries all use the printf function, the code of printf is copied inside all 10 binaries).
- **Dynamic linking**: The needed code is not copied inside the binary; instead, the binary contains information (i.e. the address of the external function or variable) used to locate the needed resource. The resolution happens at runtime, when the Dynamic Linker finds and loads the library into the process's virtual address space and resolves the references to the symbols. This allows different processes to share the same SO.

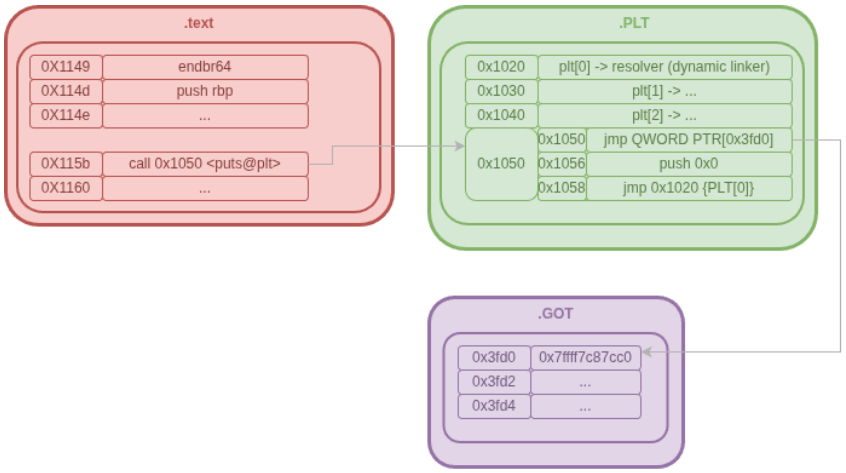
Since nowadays the most commonly used approach is **Dynamic Linking**, the following discussion will focus on it.
As stated before, the linking process can be used for both external variables and external functions. In both cases the process is the same, since both functions and variables can be exported as symbols.
When dealing with functions there is the possibility of using **Lazy Binding**. This technique consists of resolving the symbol only during the first invocation of the function, rather than at load time as happens with variables. In order to achieve **Lazy Binding** a stub section is used (i.e. **.plt**). When a program uses a library function, a PLT entry is added to the .plt section. As can be seen in fig. 1.4, the relocation is resolved lazily upon the first invocation of the function. When control is transferred to the corresponding PLT entry, the entry performs an indirect jump through its associated GOT entry. Before the symbol is resolved, this GOT entry contains an address that redirects execution back to the PLT, eventually reaching the PLT resolver stub. The resolver then invokes the dynamic linker, which performs the symbol lookup, resolves the relocation, and updates the GOT entry with the resolved address of the function, as can be seen in fig. 1.4b. Subsequent calls can therefore jump directly to the resolved address through the GOT.

1.3.3. ELF Loading

The next useful step in order to understand the chosen approach consists of going into detail on the ELF Loading phase. There are 3 main actors that allow a binary to be



(a) An example of puts function resolution using Dynamic Linking and Lazy Binding before the actual resolution



(b) Same binary as fig. 1.4a but with symbol resolved

Figure 1.4: An example of Lazy Binding involving .plt and .got

executed properly:

- **Kernel**: responsible for the actual `execve` invocation and allocation of the different memory segments. Moreover, it is the one that executes the dynamic linker used to resolve dependencies prior to the execution of the binary.
- **Dynamic Linker**: Loads all the needed libraries into the process memory, resolves the external dependencies, prepares the .got and .plt, and executes the initializers of the libraries.
- **Target Binary**: Invoked by the Dynamic Linker, it prepares the environment for the main function and, in the end, runs it.

All these procedures can be seen in detail in fig. 1.5. The dynamic linker part is the

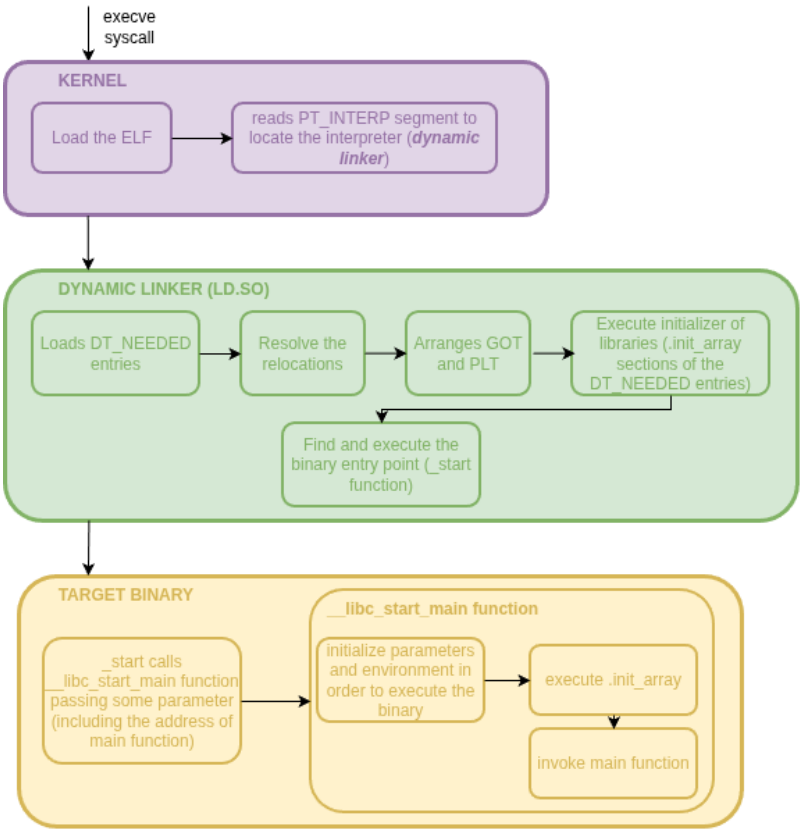


Figure 1.5: Dynamically linked ELF loading procedure

one on which I will focus in this dissertation. The dynamic linker:

- *Loads all the DT_NEEDED entries*: these are the dependencies (i.e. the SOs needed by the binary). For example, if the binary uses an external function contained in *libc.so*, the library from which the function is exported must be added as a dependency.
- *Resolves the relocations*: a focus on relocations will be presented in the next subsection.
- *Arranges the GOT and PLT.*
- *Executes all the initializer functions (constructors) for each dependency*: For each *DT_NEEDED* entry found by the dynamic linker, it executes all the functions contained in the *.init_array* section of that entry. These are functions declared with `__attribute__((constructor))`.
- *Finds and executes the binary entry point.*

1.3.4. ELF Symbols and Relocations

The dynamic linker uses symbols to link different ELFs together.

- A component (an ELF or a SO) can export a symbol representing a variable or a function inside it.
- Another component can reference the exported symbol and use it (if the first component is a dependency of the second).
- The linker acts as the intermediary between the two components, responsible for matching the symbol names (the exported one and the referenced one) and storing the address of the exported symbol in the second component's address space in such a way that this component can access it.

All the symbols belonging to an ELF (both imported and exported) are stored in the *.dynsym* section. In fig. 1.6 a list of symbols extracted from a SO can be seen. The

```
20: 0000000000000000     0 FUNC    GLOBAL DEFAULT  UND exit@GLIBC_2.2.5 (2)
21: 0000000000000000     0 NOTYPE  WEAK   DEFAULT  UND _ITM_registerTMC[...]
22: 0000000000000000     0 FUNC    WEAK   DEFAULT  UND [...]@GLIBC_2.2.5 (2)
23: 0000000000000000     0 FUNC    GLOBAL DEFAULT  UND fork@GLIBC_2.2.5 (2)
24: 00000000000040b8     4 OBJECT  GLOBAL DEFAULT   25 standalone
25: 00000000000013f2   373 FUNC    GLOBAL DEFAULT   14 aflplusplus_fork[...]
26: 0000000000001891   137 FUNC    GLOBAL DEFAULT   14 afl_forkserver_c[...]
27: 00000000000016fd   404 FUNC    GLOBAL DEFAULT   14 init
28: 00000000000040b0     8 OBJECT  GLOBAL DEFAULT   25 __forkserver_sta[...]
29: 0000000000001567   252 FUNC    GLOBAL DEFAULT   14 afl_forkserver_start
30: 0000000000001663   154 FUNC    GLOBAL DEFAULT   14 standalone_forkserver
31: 00000000000040a8     8 OBJECT  GLOBAL DEFAULT   25 __afl_area_ptr
```

Figure 1.6: Example of a *.dynsym* table for a shared object

```
Relocation section '.rela.dyn' at offset 0x7ddaac contains 10 entries:
  Offset          Info           Type           Sym. Value    Sym. Name + Addend
000000808ff8  002300000006 R_X86_64_GLOB_DAT 0000000000000000 __gmon_start__ + 0
000000809280  004000000005 R_X86_64_COPY     0000000000809280 __progname@GLIBC_2.2.5 + 0
000000809288  003f00000005 R_X86_64_COPY     0000000000809288 stdout@GLIBC_2.2.5 + 0
000000809290  004100000005 R_X86_64_COPY     0000000000809290 stdin@GLIBC_2.2.5 + 0
000000809298  004600000005 R_X86_64_COPY     0000000000809298 optind@GLIBC_2.2.5 + 0
0000008092a0  004700000005 R_X86_64_COPY     00000000008092a0 optarg@GLIBC_2.2.5 + 0
0000008092a8  004300000005 R_X86_64_COPY     00000000008092a8 __progname_full@GLIBC_2.2.5 + 0
0000008092c0  004500000005 R_X86_64_COPY     00000000008092c0 stderr@GLIBC_2.2.5 + 0
0000017db000  003d00000006 R_X86_64_GLOB_DAT 0000000000000000 __afl_area_ptr + 0
0000017db008  003e00000006 R_X86_64_GLOB_DAT 0000000000000000 __forkserver_start_ptr + 0
```

Figure 1.7: Example of a *.rela.dyn* relocation table extracted from a binary

first 4 symbols are external symbols that must be resolved. This can be seen because the

Value column is 0 and Ndx is UND (undefined). The last 8, instead, are symbols declared inside the shared object. When the visibility is set to *DEFAULT*, this means that the symbol can be visible outside the shared object.

The process of resolving symbol references is called **Relocation**. The relocation algorithm is shown in algorithm 1.1. Relocating a symbol means resolving the symbol (i.e. finding the address of that symbol in the original object), and writing that address inside the binary that has the relocation. Specifically, the symbol address is written inside the GOT. In fig. 1.7, the address at which the symbol address must be written can be seen in the "Offset" column.

Algorithm 1.1 Relocation resolution at ELF load time

Require: $L = [obj_1, \ldots, obj_n]$ {The objects that have a dependency (DT_NEEDED) must already be loaded in memory}
Ensure: for each $obj \in L$, the respective GOT entry contains the resolved runtime address of the external symbol.

1: **for** $obj \in L$ **do**
2: $Reloc_table \leftarrow obj$.`GET_SECTION(".rela.dyn")`
3: **for** $reloc \in Reloc_table$ **do**
4: **if** $reloc$.type = `R_*_RELATIVE` **then**
5: $addr \leftarrow obj$.`loadBias` + r.addend {The symbol does not need to be resolved. The address must be adjusted in order to enable ASLR}
6: **else**
7: **if** $reloc$.type $\in$ {`R_*_GLOB_DAT`, `R_*_JUMP_SLOT`} **then**
8: $sIdx \leftarrow reloc$.symbol_index
9: $symbolName \leftarrow obj$.`dynstr`[$obj$.`dynsym`[$sIdx$].name] {The name of the symbol is read from *.dynsym*}
10: $target \leftarrow$ ResolveSymbol($symbolName, L, obj$)
11: **if** $target$ = NULL **then**
12: **error**: undefined symbol $symbolName$
13: **else**
14: $(defObj, defIdx) \leftarrow target$ {*defObj* is the object that exports the symbol, *defIdx* is the index in the .dynsym table of *defObj*}
15: $addr \leftarrow defObj$.`loadBias` + $defObj$.`dynsym`[$defIdx$].value {The address of the resolved symbol is computed as base_address (of the object that exports the symbol) plus the offset at which the symbol is stored}
16: **end if**
17: **end if**
18: **end if**
19: write $addr$ in obj.`got` at offset $reloc$.offset {The destination address at which to write the resolved symbol address is specified in the relocation entry}
20: **end for**
21: **end for**

Algorithm 1.2 ResolveSymbol function used by algorithm 1.1

Require: name *name*, object list L, object requesting the resolution *requester*
Ensure: pair (obj, idx) of the first symbol found, or NULL if the symbol has not been defined by any object.

1: **for** $obj \in L$ {The order is: first the executable, then all the DT_NEEDED dependencies in order of definition} **do**
2: $idx \leftarrow$ GnuHashLookup(*name*, *obj*.`gnuHash`, *obj*.`dynsym`, *obj*.`dynstr`) {As seen before, the hashes are used in order to speed up the symbol lookup}
3: **if** $idx \neq$ NOT_FOUND **&&** *obj*.`dynsym`[idx].st_shndx $\neq$ `SHN_UNDEF` **then**
4: **return** (obj, idx)
5: **end if**
6: **end for**
7: **return** NULL

1.4. Frame pointer omission and Red Zone

The classic function preamble, inserted by the compiler at the beginning of a function, can be seen in Listing 1.1.

```
1 push rbp
2 mov rbp, rsp
3 sub rsp, 0x30
```

Listing 1.1: Classic function preamble supposing a function that uses 0x30 bytes of stack

Every function has its own **stack frame**, that is the stack zone where all the local informations used by the function lies. Specifically it contains local variables, parameters passed to the function, ... The stack frame is delimited by the value of two registers:

- **Base Pointer** : point to the base of the activation frame (high addresses)
- **Stack Pointer** : point to the top of the stack (low addresses)

When a function is invoked, the stack frame of the new function must be allocated. This is done by the Listing 1.1. The effect on the stack could be seen in fig. 1.8. If the function had a local variable, that variable would be referenced with respect to the base pointer (e.g. `[rbp - 0x8]`). The base pointer in this case is called **frame pointer** and it is used as a stable reference to the stack frame. Infact, during the execution of a function, the stack pointer could change (a `push` or a `pop` instruction occur), and this case the stack pointer would not be a valid reference to the variables anymore.
If the compiler knows a priori that the stack pointer will not change during function execution, could optimize the function preamble with a technique called **frame pointer omission**. This technique consist in not use the base pointer as reference to the local

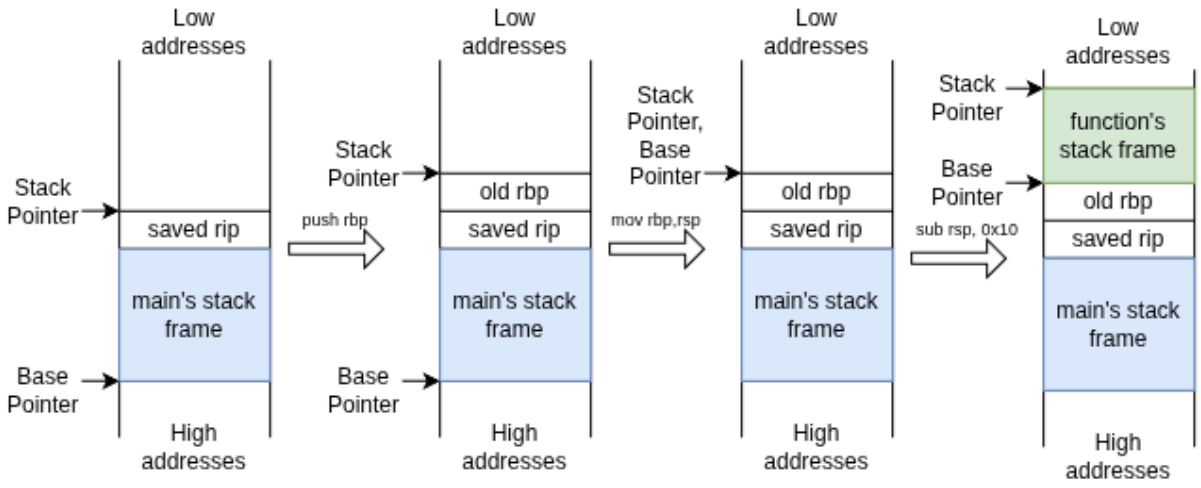


Figure 1.8: Example of what happend during a function call

variables, but using directly the stack pointer. This allow the function preamble to be shrink to the one shown in Listing 1.2

```
1 sub rsp, 0x30
```

Listing 1.2: Function preamble in case of frame pointer omission supposing a function that uses 0x30 bytes of stack

In this case the access to local variable is expressed with respect to the stack pointer (e.g. `[rsp + 0x8]`), as can be seen in fig. 1.9.

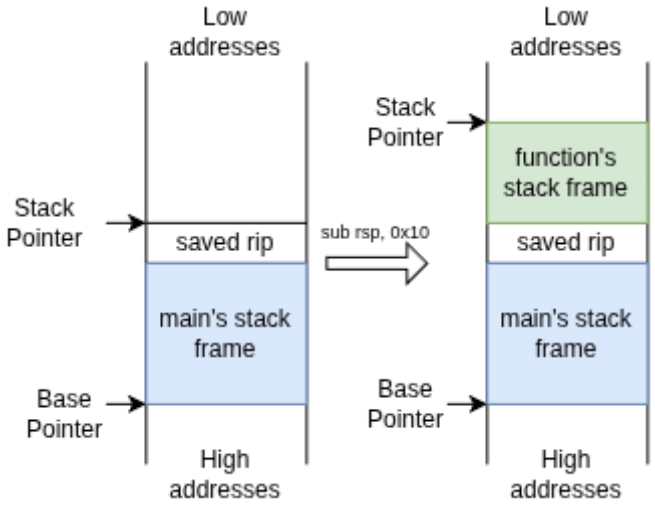


Figure 1.9: Example of what happend during a function call in case of frame pointer omission

red zone

The red zone is composed of 128 bytes above the rsp that are reserved by the ABI System V AMD64. The user code that is compliant with the ABI could use this reserved memory to save temporary data without actually modify the stack pointer and allocate a stack frame. Since the stack grows toward low addresses, this zone is overwritten by the stack frame of any function that is called from the actual one, so the red zone could be used especially for leaf functions (functions that does not call other functions).
The compiler could choose to bypass entirely the function premable and access the local variable with respect to the stack pointer (e.g.`[rsp -0x8]`) as shown in fig. 1.10

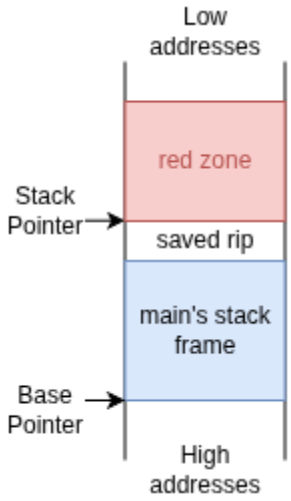


Figure 1.10: Example of function call that does not allocate space of the stack bu access the variable saved on the red zone

1.5. Problem Statement

Coverage-Guided Grey-box fuzzing is one of the most effective techniques for automatic vulnerability discovery in software [6, 26], due to its ability to generate a wide range of inputs based on the feedback obtained through Code Coverage information, which guides the fuzzer towards unexplored regions of code and therefore helps it find hidden vulnerabilities. In fact, *Black-box* fuzzing lacks a precise mutation strategy based on execution monitoring, while *White-box* fuzzing incurs high overheads due to Symbolic or Concolic execution. Furthermore, the latter approach also requires the availability of the source code in the general case, which is not always available. So *Grey-box* fuzzing seems to be the most promising trade-off between precision of input mutation and overhead. However, *Grey-box* fuzzing also has its own drawbacks. As a matter of fact, the effectiveness of this approach strongly depends on the availability of the source code. In fact, both SBI and DBI have many limitations. The main limitation of DBI is the huge amount of

overhead (higher than 600% with respect to the compile-time instrumentation in the case of AFL++ in QEMU mode [26]), which makes the exploration of a substantial part of the valid input space almost impossible. Instead, the main difficulty of SBI arises in modifying already compiled binaries. There are several problems regarding Binary Rewriting, such as Indirect Control Transfers (e.g. `call rax`), shifts in binary sections, or instructions that require rewriting the instructions that reference these sections, and many others, as seen in the previous Section 1.2.
For all these reasons, the preferred technique nowadays is *Compile-time Edge Coverage instrumentation* with fuzzers such as AFL++, LibAFL [13], or libFuzzer [21]. However, in practice, for reasons of industrial secrecy (i.e. proprietary software) or due to precompiled libraries, applications are often closed source, and so Compile-time Edge Coverage instrumentation becomes unusable.
In these scenarios, binary-only fuzzing is the only viable approach. Although SBI presents many challenges, it nonetheless proves to be the most fruitful area, given its low overheads [26]. In this thesis I will address the problem of statically instrumenting a binary with Edge Coverage instructions in a way that makes the instrumented binary compatible with AFL++, which has been chosen as the fuzzer due to its popularity and because it represents the current state of the art, implementing most of the recently discovered techniques demonstrated by Andrea Fioraldi et al. [12].
The instrumentation must add as little overhead as possible with respect to compile-time overheads and must preserve the correctness of the binary. This is important in fuzzing, because lower overhead means more executions per second, so a larger portion of the valid input space can be explored.

1.6. State of the Art

In this section I will present some of the current state-of-the-art solutions in binary-only grey-box fuzzing, distinguishing between SBI and DBI approaches.

- **DBI**
  - **AFL++ in QEMU/FRIDA mode** [12]: AFL++ in QEMU mode uses QEMU in *User-mode Emulation*, which consists in translating guest machine code (i.e. guest BBs) into an intermediate representation called Translation Block (TB) and subsequently appending the coverage tracing instructions at the end of it. This approach is based on emulating the entire CPU using QEMU's virtualized environment. As stated before, AFL in QEMU mode has overheads on the order of 600% with respect to compile-time instrumentation

[26].
AFL++ in FRIDA mode [1], instead, is based on shared library injection, in which a Frida [27] component called **Stalker** intercepts any BB that is about to be executed, copies it into a **Code Cache** and dynamically instruments the copy with coverage tracing instructions. Execution control is then transferred to the copy, which is executed and then re-intercepted by **Stalker** at the end. Unlike QEMU mode, it does not emulate the CPU, but instead executes within the memory space of the target program. AFL++ reports that FRIDA mode is approximately 2 to 5 times slower than compile-time instrumentation [2].

- **DynamoRIO/Pin** [4, 22]: Both these approaches share the same basic principle, called **Just in-time (JIT) code caching**. This consists in taking control of the thread when the target application starts, and from that moment on, every BB is intercepted before it is executed and is copied into a private **Code Cache**. The instrumentation is then added to the copy, and control is transferred to the code cache. The instrumentation process is more or less identical to FRIDA's Stalker mechanism, but is not as optimized. Also in this case, the implemented version uses AFL [34] as the fuzzer [18, 19].

- **SBI**

  - **Retrowrite** [9]: This tool performs symbolic lifting on the code, replacing every reference discovered during disassembly with a symbolic label. Doing this allows instructions to be added without modifying the references, achieving reassembleable assembly. While RetroWrite can add ASan instrumentation, it cannot natively add AFL-style edge coverage instructions. To do so, the lifted assembly must be instrumented with the afl-gcc tool in order to produce the patched binary. Moreover, community-reported issues [24] document missing instrumentation points when RetroWrite's symbolized output is compiled through afl-gcc, caused by a mismatch between RetroWrite's label-naming convention and afl-as's basic-block detection heuristics. Finally, RetroWrite only works with PIE, non-stripped binaries.

  - **AFL-Dyninst** [17]: AFL-Dyninst employs Dyninst's binary rewriting capabilities to instrument an existing executable by inserting callbacks at selected basic blocks. Dyninst's instrumentation mechanism is based on code patching and trampoline redirection [5], which causes many context switches and state saving/restoring operations. Its performance is slightly better than DBI, but this approach still reaches 500% fuzzing overhead, as demonstrated by Stefan

Nagy et al. [26].

- **E9AFL** [15] : E9AFL uses E9Patch [11] to instrument binaries. This is a framework that allow to modify existing isntruction without actually retrieve and analyze the whole CFG. E9AFL modify existing instruction inserting trampolines to a code region that update the Shared Memory (SHM). Since trampoline based instrumentation suffer of many overheads, this approach introduce many optimization techniques that reduce the overheads given by the trampolines. Alvin Charles et al. [7] shows that E9AFL has some limitations, especially regarding the capability of instrumenting correctly the target binaries.
- **ZAFL** [26]: This tool uses two main phases: the first consists in reconstructing an IR using the Zipr binary rewriter. The resulting IR is then used by ZAX to perform an optimization on the IR, find the instrumentation points, and finally instrument these points. ZAFL works by inserting inline calls to a function that updates the shared map at the beginning of each BB. Despite ZAFL's many improvements, Jifan Xiao et al. [32] demonstrated that it still causes crashes in 28% of the binaries used in their cited paper.

Before proceeding to the next section, in which I will discuss the main goals of this thesis, I want to discuss the core functioning of AFL and AFL++, given its importance for understanding my approach.

1.6.1. AFL Functioning

[33] As previously stated, AFL and, in particular, AFL++ are among the most widely used grey-box fuzzing frameworks [12]. AFL++ represents the community-based continuation of the original AFL project. The core functioning is virtually the same and consists of a SHM region shared between the fuzzer and the target. The SHM size can vary, but the default is 65536 bytes.
Each "cell" of the SHM contains the number of times a given edge in the CFG has been taken during the execution of the target. Since the SHM has 65536 "cells", each "cell" can contain a number from 0 to 255. The "cell" position is computed with a specific hash function applied to the IDs of the two BBs that form the edge. The main Edge Tracing update rule is summarized in algorithm 1.3. Line 3 of algorithm 1.3 is used to avoid updating the same SHM location when an edge is traversed forward or backward. An example of this could be a simple `do-while` loop, as seen in algorithm 1.4 and fig. 1.11.
The main idea behind AFL is to instrument each basic block of a target application with

Algorithm 1.3 AFL-style SHM update rule

1: $idx \leftarrow current_bb_id \oplus previous_bb_id$
2: $shared_map[idx] \leftarrow shared_map[idx] + 1$
3: $previous_bb_id \leftarrow current_bb_id \gg 1$

Algorithm 1.4 Do-While Example

repeat
{Basic Block A}
until *Permanence_Condition*
{Basic Block C}

Without 3 of algorithm 1.3, the hash of Edge1 would result in $ID_A \oplus ID_B$, and the hash of Edge2 (if the permanence condition is not met) would result in $ID_B \oplus ID_A = ID_A \oplus ID_B$, due to the symmetry of XOR. But since the hash is the same, there would be a hash collision that would cause the same "cell" to be updated twice instead of once.

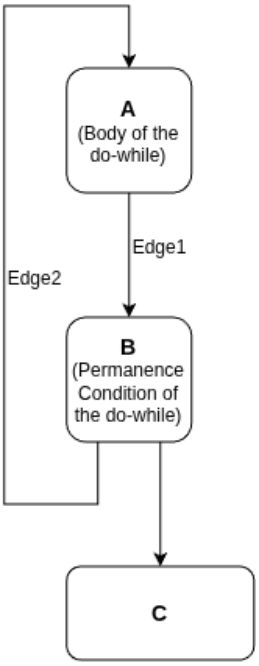


Figure 1.11: Do-while CFG

the shared map update rule. The fuzzer initializes the SHM memory with zeros, runs the instrumented target, which updates the SHM according to algorithm 1.3 until it reaches the end, and then returns to the fuzzer. The SHM ID is communicated to the target through an environment variable called $__AFL_SHM_ID$.
At this point the fuzzer checks the SHM it has just obtained, called `trace_bits`, searching for any newly discovered edge. This is done by keeping a structure called `virgin_bits`, which contains all the edges not yet discovered and is initialized with ones. The comparison is performed through `trace_bits` $\wedge$ `virgin_bits`. If any position results in one after the logical AND, this means a new edge has been discovered. In this case, the `virgin_bits` data structure is updated and the input just executed is saved in the corpus. Otherwise, a new input is chosen, mutated and then executed.
This approach requires a `fork()` call followed by an `execve()` call for each input, causing significant overhead for each input execution. Two main optimizations have been implemented:

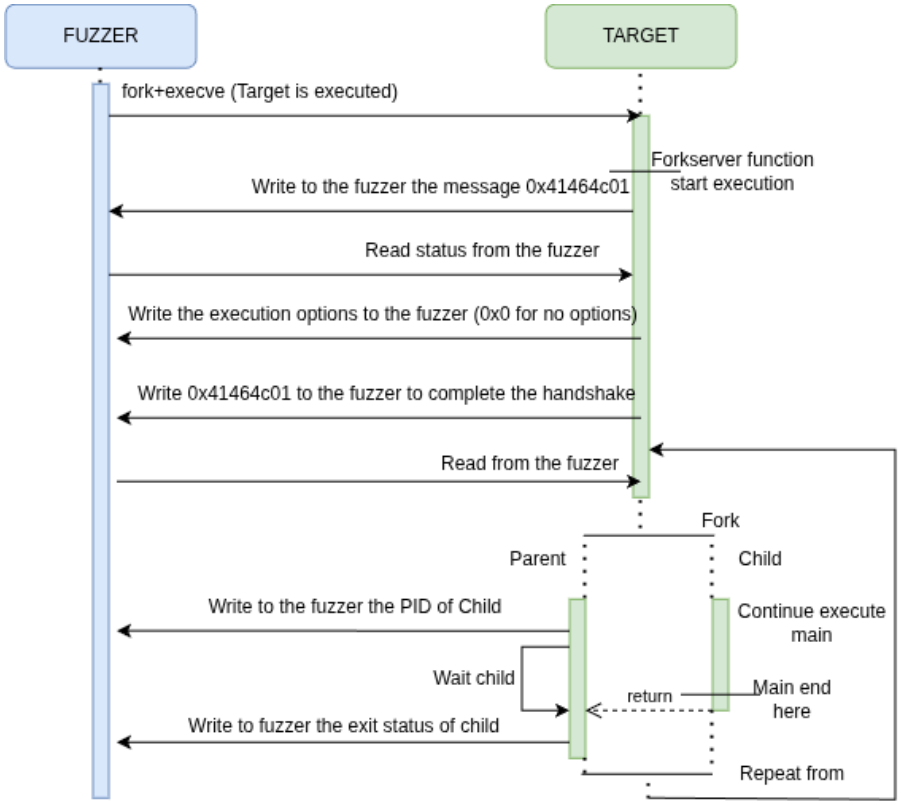
- **forkserver**

- **persistent mode**: (not shown in this thesis)

Forkserver

The forkserver makes it possible to avoid calling **execve** for every input, performing it only once at the start of execution. The forkserver is instrumented before the main function of the target application. The AFL++ forkserver protocol can be summarized as follows: [2]

Figure 1.12: AFL++ Forkserver Protocol



1. The forkserver sends the "Welcome" message to the fuzzer (0x41464c01).
2. The fuzzer replies with the status code. At this point the forkserver must check whether $status \oplus 0xffffffff = 0x41464c01$. If it is equal, execution continues; otherwise, the forkserver stops here.
3. The forkserver sends the execution options to the fuzzer. These can indicate that the target is running in persistent mode, or other optimizations.

[2] *The differences between the AFL forkserver protocol and the AFL++ forkserver protocol lie mainly in the messages exchanged between the forkserver and the fuzzer*

4. The forkserver sends 0x41464c01 to the fuzzer to end the handshake.
5. The fuzzer writes a message to the forkserver (its content is discarded); the handshake is now finished. This message is the "go" signal for the forkserver.
6. The forkserver invokes `fork()`. The parent writes the PID of the child to the fuzzer and then invokes `waitpid()`.
   The child continues execution of the main function until it ends.
7. When the parent returns from the `waitpid()` function (i.e. the child has terminated), the exit status of the child is sent to the fuzzer, and the forkserver waits for another "go" from the fuzzer.

The communication between the Fuzzer and the Target shown in fig. 1.12 occurs using two unidirectionally pipes that uses the file descriptor:

- *FORKSRV_FD (fd 198)*: used by the forkserver to read a message sent by the fuzzer.
- *FORKSRV_FD + 1 (fd 199)*: used by the forkserver to write a message to the fuzzer.

An approach worth mention in this section is the *deferred forkserver* technique. It consist in execute the forkserver as later as possible in the target. As shown previously, at each input, all the instructions after the forkserver must be executed by the child. If the forkserver is invoked before some instructions that does not require tracing (for example the load of a configuration file, or the initialization of an object), these instructions are executed for each input, producing useless overheads.

1.7. Goals and Challenges

The problem that I want to address in this thesis is to develop a tool capable of inline, static edge-coverage instrumentation that preserves binary correctness while achieving better performance with respect to DBI and comparable to other state-of-the-art instrumentation technique.
I have chosen to implement the SBI approach instead of DBI due to its low overhead. To do this, I will use the GRAPE binary rewriting framework, a novel approach for binary rewriting that preserves binary executability while introducing small overheads compared to a trampoline-based approach. This approach involves several challenges, mainly related to preserving the binary's correctness and behavior. Moreover, the tool must be compatible with the already existing AFL++ fuzzer. This means implementing the forkserver

communication protocol inside the target binary without recompiling it. This has created no small number of problems, mainly regarding the preservation of the ELF layout after adding new sections.

The implemented approach drastically increases the size of the file, caused by either instruction repetitions (since the SHM instructions are always the same), and a constant factor given by the new added sections, but was chosen because placing instructions inline it cause less overheads with respect to a call. In fact, in order to call a function, the state of the registers must be saved and restored. Moreover, the stack must be allocated, and the callee's RIP must be saved on the stack alongside the old base pointer of the stack. Performing all these operations at each BB would introduce significant overhead.

The main contribution of this work consist in creating an "extension" of GRAPE framework capable of instrumenting edge coverage instructions comparable with AFL++, or rather an application almost entirely based on GRAPE that seeks to investigate the potential use of GRAPE in the context of fuzzing.

2 | Approach

In this chapter I will disclose the general aspects of the approach regarding the insertion of the Edge Tracing instructions inside an already compiled binary.
Four are three main steps to be fullfilled to have a binary that is able to trace Edge Coverage and to be compatible with AFL/AFL++:

1. Insertion of the forkserver invocation prior the main (section 2.1).
2. Shared Memory address discovery (section 2.2).
3. Search of instrumentation points to trace Edge Coverage.
4. Intrument the previously found points with Edge Coverage tracing instructions.

2.1. Forkserver Insertion

This has been one of the most crucial points of the whole work. First of all I have tried to instrument the forkserver function directly inline. This approach presents many problems. The biggest difficulty is the fact that a forkserver needs some library functions to work properly:

- `read/write` to implement the communication protocol with the fuzzer.
- `fork/waitpid` to implement the forkserver main logic.

These functions could be part of the target binary (if the binary already uses these functions) or not. In the best case, the binary already uses these functions and they are available. In this case, a call to these functions could be instrumented at any time, taking into consideration the type of linking with which the binary has been compiled.

- In case of *Static Linking*, the function instructions are already inside the target application, because the linker inserted them during compilation. So a call to these functions could be inserted by finding the function address and then instrumenting a call instruction to that address.
- In case of *Dynamic Linking*, the function instructions are not inside the target

application when it is launched, but the function addresses are dynamically resolved by the dynamic linker as needed. In this case, two new sections must be used (the `.plt` and `.got` sections, as seen in section 1.3).

If the functions required by the forkserver are part of the target binary, their calls can be easily instrumented. Depending on whether the binary is statically or dynamically linked, this can be done either by finding the address of the function and calling it directly, or by finding the address of its .plt entry and calling that, leaving the dynamic linker to resolve the function. The problem arises when the original binary does not include these functions. In this case, it is almost impossible to modify the .plt and .got sections to include the new functions while preserving the binary's layout and correctness in the general case. In fact, in some exceptional cases padding may provide enough space in the .got section for a new entry, but this cannot be assumed in general. Therefore, this first approach has been completely discarded. Since the forkserver is more complex than a simple shared map update, due to the presence of many checks on environment variables and communication with the fuzzer, the best approach is to inject the compiled forkserver code into the target through *Shared Object Injection*. In order to achieve Shared Object Injection, the *.dynamic* section of the target binary must be modified. In particular:

- A *DT_NEEDED* entry is added with the name of the shared object to be injected. [1]
- A *RUNPATH* entry is added containing the path where the actual shared object is saved.

[2]

The shared object injected into the target binary contains the code of the forkserver, which must be executed before the main of the binary. In order for a function of a shared object injected into a binary to be executed before the main, that function must be declared as a "constructor". As stated in section 1.3, by declaring a function as a constructor, its address is placed inside the *.init_array* section and executed before the actual start of the executable's main. When an executable has some dependencies, the dynamic linker executes the *.init* section and the functions contained in the *.init_array* section before actually starting the main of the executable.[3] In fig. 2.1b the execution flow

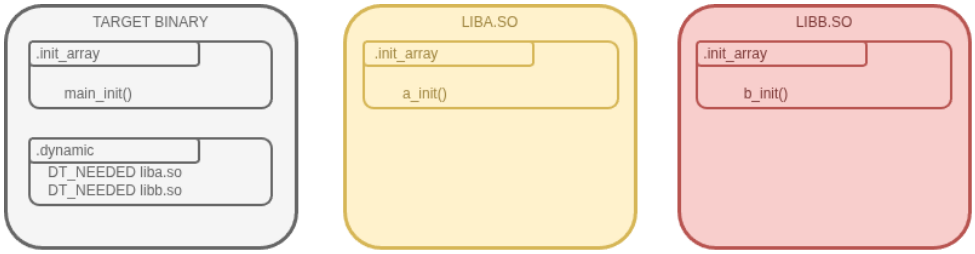
[1] In reality the shared object name is a string that is written into the *.dynstr* section, and the *.dynamic* section specifies the offset of the string written into *.dynstr*.

[2] The injection could also be done with the LD_PRELOAD environment variable. This variable tells the dynamic linker to add the specified libraries as dependencies of the executable. The problem is that at each execution the variable must be reset with the name of the shared object. By adding the DT_NEEDED dependency instead, the dependency becomes persistent.

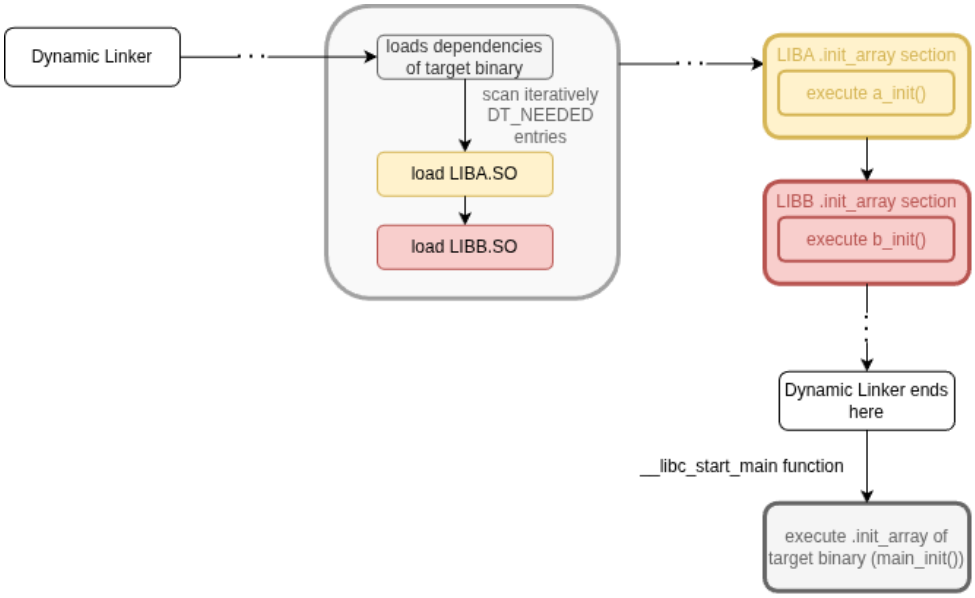
[3] The .init section is the legacy version of the .init_array section.

of the dynamic linker can be seen as it initializes the dynamic dependencies of a target binary that has the dynamic dependencies shown in fig. 2.1a.

1. The dynamic linker searches for *DT_NEEDED* entries of the target binary. When a dependency is found, it maps the library into a loaded Virtual Memory Area (VMA) of the process associated with the target binary being executed.
2. For each library found previously, the dynamic linker executes each function found in the *.init_array* section of that shared object.
3. When the dynamic linker finishes, the `_start` function is executed, and it in turn executes `__libc_start_main`, which executes all the functions in the *.init_array* of the target binary.



(a) Layout of a target binary with its dynamic dependencies



(b) Dynamic linker initialization of dynamic dependencies

Figure 2.1: An example of ELF execution with dynamic dependencies and a constructor function

Therefore, injecting a SO that has a function declared as constructor into the target binary could resolve the problem of instrumenting the forkserver so that it is executed

before the main.

2.2. Shared Memory Instrumentation

As stated in the previous chapter, the SHM is declared by the fuzzer and the forkserver can know the ID of the SHM thanks to an environment variable set by the fuzzer. The forkserver is injected as a constructor function of a SO for which a DT_NEEDED dependency has been added to the target binary. But, as stated before, the dynamic linker executes the constructor of the SO (and therefore the forkserver) before the actual initialization of the binary. Moreover, the SHM address is saved in a variable of the SO and is not, by default, visible inside the target binary. The best approach that allows a variable to be accessible outside a binary is to export the symbol that represents the variable.

As seen in section 1.3, the relocations for external symbols are resolved during the execution of the dynamic linker before the main (or at the first invocation of the function in the case of lazy binding). Since the SO contains the symbol holding the address of the SHM, the idea to let the target binary know the address of the SHM is:

- The SO exports that symbol, declaring it with `__attribute__((visibility("default")))` [4]
- Add a relocation entry to the target binary.

By doing this, the dynamic linker, at binary loading, will resolve the relocation, finding the address of the variable that stores the address of the SHM inside the SO, and write it into the *.got* of the target binary.

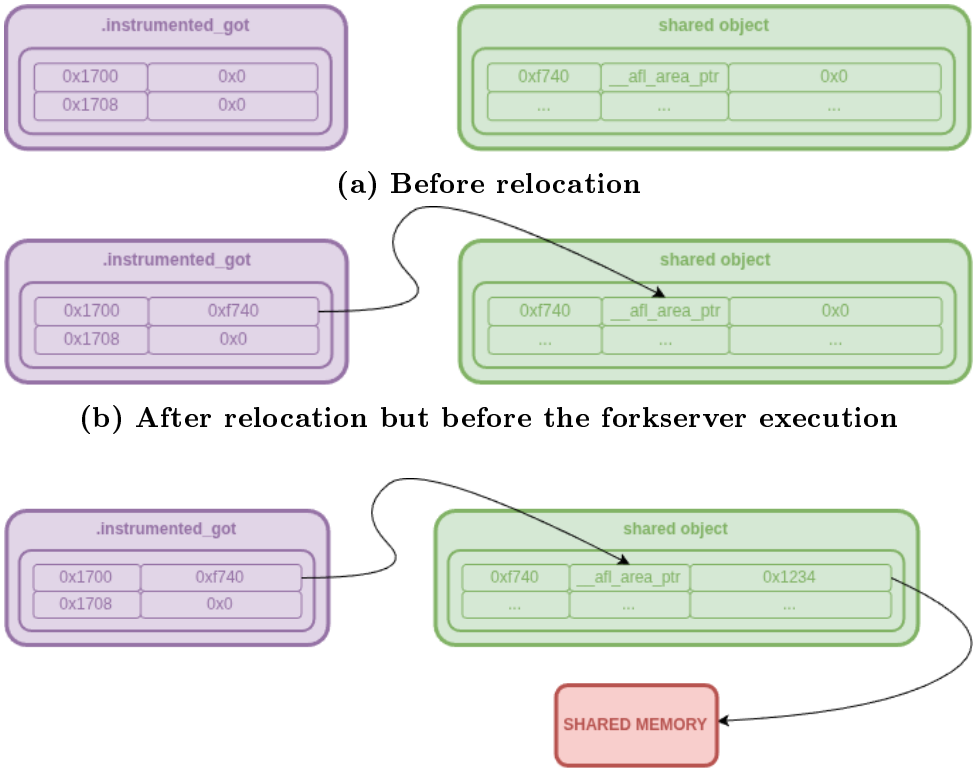
Since, as stated in section 2.1, it is difficult to add a GOT entry in the general case (because it depends on the presence of available space in the GOT), the chosen approach has been to add a new section to the binary where the needed symbols, after relocation, will be written. Now the complete method to make the SHM accessible to the target instrumented binary can be reviewed:

1. Add a new section to the binary. This section will be needed as a GOT in which the resolved symbol values will be written.
2. Add a dynamic symbol to the binary. This is the local symbol that will be referenced by the relocation table. The name of the local symbol must be the same as the symbol exported by the SO that has already been injected as a dependency of the

[4] It is not mandatory to explicitly write the visibility attribute, because each symbol has "default" visibility by default.

binary.

3. Add a dynamic relocation entry. This relocation entry must link the symbol to be relocated with the memory address at which the resolved value will be written (i.e. the section previously created to host the GOT).



(a) Before relocation

(b) After relocation but before the forkserver execution

(c) After forkserver execution

Figure 2.2: Shared Memory relocation between the target binary and the Shared Object

According to fig. 1.5, the relocations are resolved before the forkserver invocation (because the forkserver lies inside the constructor of the shared object). The correct value for the SHM address is obtained by the forkserver, but the symbol is resolved beforehand, and therefore the address of the symbol is written inside the GOT before the variable contains the address of the SHM. When the main starts, the forkserver has already been executed, and dereferencing the relocated symbol will result in the correct value initialized by the forkserver. The process can be seen in fig. 2.2. In fig. 2.2a the GOT contains the address 0x0, because the relocation has not yet been performed (assume this is after the SO loading but before symbol relocation). Just after the relocation of symbols is performed, the GOT will contain the address of the external variable, but that variable will still contain 0x0, because the SHM has not been attached yet, as shown in fig. 2.2b. After

forkserver execution, the external variable will also contain the address of the SHM, as shown in fig. 2.2c.

2.3. Instrumentation Points Search

At this point the target binary has been modified with:

- A DT_NEEDED entry has been added to *.dynamic*, pointing to a shared object that contains the forkserver declared as a constructor, alongside a RUNPATH that points to that shared object.
- A symbol representing the pointer to the SHM has been inserted into *.dynsym*.
- A new section that will be used as a personalized GOT is added.
- A relocation that links the symbol to be resolved to the new section is added.

Now the IPs must be identified so that they can subsequently be instrumented. First, a distinction must be made between two types of instrumentation:

- An initial instrumentation is needed in order to initialize some variables needed for the SHM update rule.
- All the other instrumentations perform the actual SHM update.

In the first case, the IP must be as close as possible to the `main` function of the binary. Finding the main address is easy for both stripped and non-stripped binaries, so, once the main address is identified, the tool tries to find the first address after the function preamble. This goal is not always achieved, because there is no unified grammar for the function prologue. It depends on how many registers the callee wants to save on the stack, how many variables the specific function needs ... In general, the first IP lies in the first BB of the main function.
All the other IPs are found with a Depth First Search (DFS) starting from the initial main BB, searching among the successor BBs of the current one. Specifically, once a BB is found and has not already been analyzed or discovered before, it is added to a stack. The IPs discovery procedure can be seen in algorithm 2.1.
Two different lists must be used in order to distinguish between found basic blocks and basic blocks to be instrumented. Moreover, if a basic block ends with a call, the next BBs are not instrumented. This is done in order to reduce the number of IPs found and so reduce the overheads of the instrumentation. For this reason I have choose to avoid instrumenting the calls, loosing only little informations, but gaining performances.
The ExtractSuccessors function will find all the successors of a BB, excluding :

Algorithm 2.1 Instrumentation point search with DFS algorithm

```
1:  found ← [ ] {found is a list of discovered basic blocks. It contains both the basic blocks
    to be instrumented and the basic blocks for which instrumentation is not required}
2:  to_instrument ← [ ] {this is the list of basic blocks to be instrumented. It is a subset of
    the "found" list}
3:  current = main_bb
4:  stack ← [ ]
5:  successors ← ExtractSuccessors(current, found)
6:  for s ∈ successors do
7:      stack.push((current, s))
8:  end for
9:  while stack is not empty do
10:     current_edge ← stack.pop()
11:     prev ← current_edge[0], curr ← current_edge[1]
12:     if prev.last_instruction._opcode ≠ "call" then
13:         to_instrument.append(curr) {If the previous basic block ends with a call instruction,
            the current basic block will not be instrumented}
14:     end if
15:     found.append(curr)
16:     successors ← ExtractSuccessors(curr, found)
17:     for s ∈ successors do
18:         stack.push((curr, s))
19:     end for
20: end while
```

- The successors that are outside the *"code"* region (so avoiding instrumenting *.plt* stubs). For *"code"* region is intended all the instruction of the main and other functions that are callable from the main. So avoid instrument functions in *.fini_array* and other not intereseting part of *.text* section.
- The edges that are already been found (and so are contained in "found" list)
- The edges that are temporarily inside the stack data structure

For example, if at a certain point the binary performs a call to a library function (e.g. `printf`), without excluding this case the program would also instrument the *plt* stub of the library function.

In fig. 2.3 a disassembled CFG obtained with Cutter [28], representing a program that computes the power operation, can be seen. With respect to fig. 2.3a, the IPs found are:

- ***0x11c2***: initial main BB. This is where the initial instrumentation will be placed.
- ***0x11f1*** (prev=*0x11c2*,next=*0x11f1*)
- ***0x11db*** (prev=*0x11c2*,next=*0x11db*)

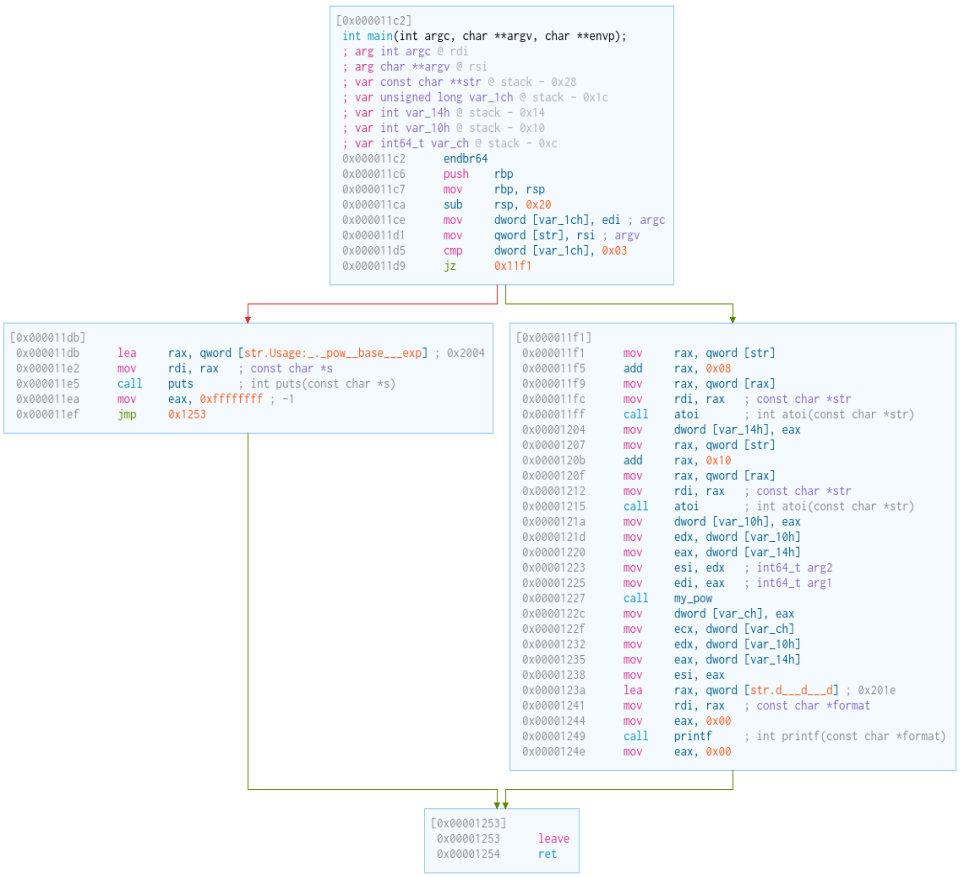
- ***0x1253***
  - (prev=*0x11f1*,next=*0x1253*)
  - (prev=*0x11db*,next=*0x1253*)

Apart from the call to the `my_pow` function, all the other functions are exported from libc, and so do not need to be instrumented.

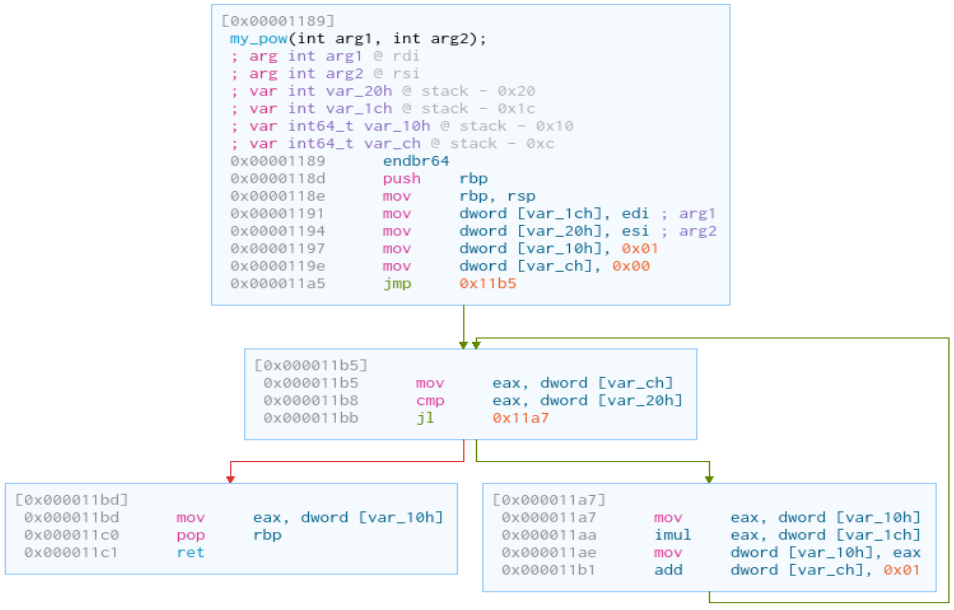
With respect to fig. 2.3b, the IPs found are:

- ***0x11b5***
  - (prev=*0x1189*,next=*0x11b5*)
  - (prev=*0x11a7*,next=*0x11b5*)
- ***0x11a7*** (prev=*0x11b5*,next=*0x11a7*)
- ***0x11bd*** (prev=*0x11b5*,next=*0x11bd*)

The initial BB of the function is not instrumented.



(a) Main CFG



(b) Pow function CFG

Figure 2.3: Example of the CFG of a program that computes the power operation between two numbers

2.4. Actual Instrumentation

In the last section, the method by which the IPs are found was discussed. In this section the actual instrumentation is reviewed. As stated in section 2.3, there are two types of instrumentation:

- The first one is used to set up the variables used by the SHM update rule.
- All the other IPs are used for the actual SHM update.

With reference to algorithm 1.3, there are 3 main variables to consider:

- *current_bb_id*: ID of the BB in which the instrumentation is placed
- *previous_bb_id*: ID of the BB from which the branch is taken
- *shared_map*: Address of the SHM

In the last example of the previous section (fig. 2.3) the IPs have already been found. Without considering the first one (which is where the initial instrumentation is placed), for all the other IPs the initial address of the BB is where the instrumentation will be placed. Let us consider, for example, the **0x11f1** IP, with previous BB equal to *0x11c2* and current BB equal to *0x11f1*. In this case, since there is only one incoming edge to **0x11f1**, the *previous_bb_id* can be placed inline. But in the general case (e.g. **0x11b5**) there are multiple edges that end in the BB. For this reason, it becomes impossible to place the ID of the previous BB directly inline. The solution implemented in the tool discussed in this thesis consists of storing a variable in a section of the binary.[5] This variable represents the *previous_bb_id* and is initialized to a random value by the initial instrumentation.

For the *current_bb_id*, instead, there is no need to store it in memory. It will be hardcoded (as a random value) in the assembly instrumentation inserted within the basic block.

The *shared_map* has already been resolved and written in the section added to the binary specifically to store resolved symbols.

So now the whole instrumentation phase can be reviewed in greater detail.

Initial Instrumentation

This instrumentation is needed to:

[5] This section could be the same one used for relocation or a new one added specifically to store the data.

- Initialize the *previous_bb_id*. This value is stored in a specific section created for the purpose of holding temporary values needed for instrumentation.
- Dereference the resolved SHM address. This initialization step is performed for optimization purposes. As can be seen in fig. 2.2, the instrumented section that works as a GOT holds the address of the variable inside the injected SO. In order to obtain the actual SHM address, the value must be dereferenced twice. If this double dereference had to be performed at every IP, it would result in a lot of useless overhead. For this reason, in the initial instrumentation the value is dereferenced twice and then saved in the same section as *previous_bb_id*. By doing this, at each IP the SHM address only has to be loaded into a register, and the offset of the correct cell computed.

SHM update instrumentation

algorithm 2.2 shows the general approach to the SHM update. Since the value of the `rax` register is modified, it must be saved before the actual SHM update and restored afterwards. The RFLAGS are automatically updated during any arithmetic operation. Since the binary behavior must be preserved, it is not possible that the RFLAGS are modified by the inserted instrumentation. For example if the operation before the instrumentation gives an Overflow, the Overflow flag must be correctly preserved to 1. Performing the add instruction (or the xor) the Overflow flag would be resetted to 0, creating a potential bug. The value of `rax` is not saved on the stack but in the same section where

Algorithm 2.2 Pseudo-code of Shared Memory update instrumentation

1: Save the state of `rax` register that will be used for computation
2: Save the RFLAGS that will be modified by the instrumentation
3: `rax` $\leftarrow$ *previous_bb_id*
4: `rax` $\leftarrow$ `rax` $\oplus$ *current_bb_id* {`rax` will contain the result of the hash operation between the basic blocks}
5: `rax` $\leftarrow$ `rax` + *shared_map_address*{since the shared map is an array, the offset is obtained by adding the initial address to the index}
6: [`rax`] $\leftarrow$ [`rax`] + 1 {the [ ] is the dereference operator}
7: *previous_bb_id* $\leftarrow$ *current_bb_id* $\gg$ 1
8: Restore RFLAGS to their values before the instrumentation
9: Restore the value of `rax` register to the previously saved value

previous_bb_id and *shared_map_address* are stored. The reason behind this choice is to avoid any variable overwriting in case of **red zone**, or displacement in case of **frame pointer omission**. Infact in some functions the compiler could choose, for optimization reason, to avoid the function preamble and reference the local variables directly from the stack pointer (see section 1.4). In these cases, if a push occur, the stack pointer will

change and the reference to the local variables will be destroyed or, worst, the push of a value on the stack could overwrite a local variable present in the red zone.
Now the complete approach can be reviewed:

- Add a dynamic dependency on a SO that contains the forkserver function.
- Add a symbol that will contain the SHM address.
- Add two new sections. The first will be used as a GOT to relocate the symbol. The second will be used as temporary memory space to store *previous_bb_id* and *shared_map_address*, alongside register saving.
- Add a relocation entry in order to resolve the symbol previously added. That symbol is exported from the SO.
- Find the IPs with the algorithm discussed in section 2.3.
- Add the initial instrumentation used to initialize the values of *previous_bb_id* and *shared_map_address* stored in the added section.
- For each previously found IP, add the SHM update instructions as seen in algorithm 2.2.

Before discuss the actual implementation of the approach, a discussion on the RFLAGS must be made, in order to better understand the design choices that will be discussed in section 3.2

2.4.1. RFLAGS

First of all is useful to know which RFLAGS could be modified by the instructions instrumented. The RFLAGS are divided into :

- *Status Flags* : Automatically modified by the arithmetic instructions
- *Control Flags*
- *System Flags* : Control the Operating System behavior, the interruption and debugging

Among these the only flags that could be modified by the instrumentation are the *Status Flags*.

In table 2.1a all the not shown bit are either Control Flags, System Flags or reserved bit for future use. Moreover only the first 32 bits are shown. All the bits from 33 to 63 are kept reserved and not used.

(a) RFLAGS composition

RFLAGS index	31-12	11	10	9	8	7	6	5	4	3	2	1	0
RFLAGS content	–	OF	–	–	–	SF	ZF	0	AF	0	PF	1	CF

(b) Status Flag meaning

Flag name	Flag meaning
OF (Overflow Flaf)	Set to 1 if the last arithmetic operation gives overflow
SF (Sign Flag)	Contain the most significant bit of the result. Show if the result is positive or negative
ZF (Zero Flag)	Set to 1 if the result of the last operation is exactly 0
AF (Auxiliary Carry Flag)	Indicate the carry from 3-th to 4-th bit
PF (Parity Flag)	Rarely used. Indicate if the least significa byte of the result contains an even number of 1
CF (Carry Flag)	Set to 1 if the last arithmetic operation has a carry.

Table 2.1: RFLAGS register content and meaning

2.5. Forkserver Optimization

Since fuzzing is a technique that relies heavily on execution speed for each single input, optimizing the instrumentation can save time. In the previously discussed approach there is a major cause of overhead: the point at which the forkserver is executed. In fact, as can be seen in fig. 1.5, since the forkserver is executed by the dynamic linker before the execution of the actual target `_start` function, for each input the target binary must execute many operations before actually reaching the main:

- The _start function must be executed.
- The __libc_start_main function must be executed, in which the environment is set up for the main execution.
- The .init_array functions are scanned and, if present, executed.

The ideal approach is to execute the forkserver at the very beginning of the main function, avoiding the execution of all the preamble functions for each input. This is, in principle, the same concept exposed in section 1.6 when the *deferred forkserver* mechanism was showed, with the difference that, in this case, the forkserver entry point is always fixed at the beginning of the main. To achieve this, the relocation mechanism is used again. In the injected SO, the constructor function no longer contains the forkserver, but is used only to prepare its execution. The workflow can be seen in fig. 2.4. Precisely, the constructor function will:

- set up the SHM, reading the SHM ID from the environment variable and assigning its address to the exported symbol.[6]

[6]In the next chapter the SHM attachment will be discussed in depth. In fact, the SO also includes a *standalone mode* in which there is no need for a SHM.

- assign to a symbol (which will also be exported) the address of the forkserver function that will be executed.

Two symbols will be added to the target binary instead of one:

- The first is the usual SHM address.
- The second is a pointer to the forkserver function.

These two symbols are then resolved by the dynamic linker at binary startup and their values are written to the section that has been added. In the initial instrumentation, the address of the forkserver function is first dereferenced and then executed. When the forkserver exits, the rest of the main function is executed, without re-executing all the binary startup functions for each input.

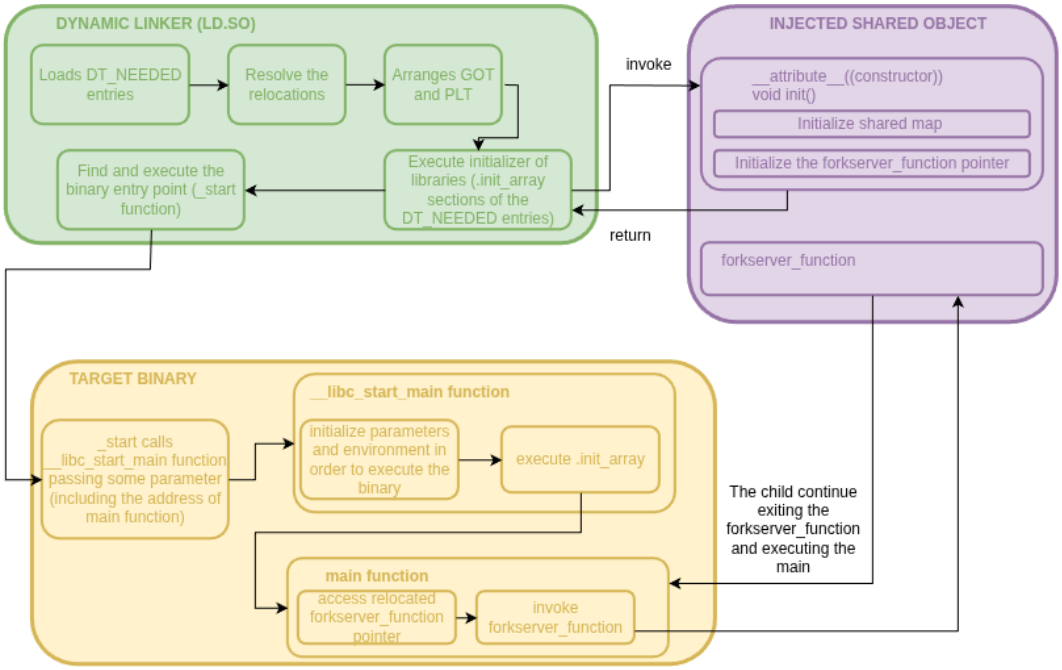


Figure 2.4: Optimized forkserver invocation

2.6. Dummy Shared Memory

The last piece of the puzzle to be added to complete the approach specification is the implementation of the *dummy shared memory*. This is a temporary memory region used mainly to prevent the program from crashing when certain circumstances are met.
In particular, especially in stripped binaries, it is difficult to correctly identify which functions are the actual program code and which functions are used by default to make the program run (e.g. *_start, frame_dummy ...*). For this reason, it sometimes happens

that some functions that are executed prior to the main are actually instrumented by the tool. Since each IP holds the instructions that perform the SHM update according to algorithm 2.2, at each IP there is an access to the *shared_map_address* saved into the created section. However, as seen in section 2.4, that address is actually written only when the Initial Instrumentation is reached at the beginning of the main.
The idea is then to create a new empty section and write the address of this memory region in the location where the shared memory address should be written. In this way, if at any point prior to the main some instruction tries to access the shared memory address, it will actually access the address of the dummy shared memory and update a memory region that is not used, so the binary does not crash. At the beginning of the main, the actual address of the SHM is dereferenced, copied from the GOT, and overwrites the address of the dummy shared memory. From this moment on, each BB updates the real SHM.
Depending on whether the binary is Position Indipendent Executable (PIE) or not, the address of the dummy section must be written differently:

- **PIE**: in the presence of Address Space Layout Randomization (ASLR), the address at which the binary will be loaded is not known a priori, and so it is impossible to know in advance the address at which the dummy section will be loaded. For this reason, a relocation must be added in order to write the correct address at load time.
- **non-PIE**: in this case the correct address is known at link time, so it is possible to write the address of the dummy section directly in the location where the SHM address will be written.

3 | Implementation

In the following chapter, the previously discussed approach will be expanded to include the technical specifications and implementation details used to achieve the described solution. Specifically two different files will be examined:

- **libforkserver.so** : this is the SO that will be injected. During the discussion a focus on the forkserver functioning will be faced, alongside the different forkserver functioning modes, the parameters and so on ...
- **instrument.py** : this is the file that actually perform the binary modification and instrumentation. In this case the details on the instrumentation, togheter with API specification, will be discussed.

3.1. Forkserver Specification

As already stated, the forkserver lies inside a SO that is injected as a dynamic dependency of the binary. Moreover, as seen in section 2.5, the pointer to the forkserver function is initialized (together with the SHM pointer) in the init function (declared as a constructor and executed before the target binary). During the execution of the "init" function, the shared object accesses two environment variables:

- $__AFL_SHM_ID$: represents the SHM ID set by the fuzzer (as seen in section 1.6).
- $__AFL_PLUSPLUS$: this is used to understand which version of the AFL fuzzer is being used. The tool supports both the AFL and AFL++ communication protocols, so this environment variable is used to choose between the two.

Depending on the value of these two environment variables, 3 different forkserver functions can be used, as can be seen in table 3.1.

- **standalone mode**: this is a mode in which the fuzzer is not present. It allows the binary to be run only once (without the typical forkserver loop) and without any communication protocol. In this case there is no SHM, so the piece of memory that

Table 3.1: Different execution modes of the forkserver based on environment variable values

__AFL_SHM_ID	__AFL_PLUSPLUS	forkserver execution mode
NULL	NULL	*standalone mode*
NULL	*any value*	*standalone mode*
NOT NULL	NULL	*afl++ mode*
NOT NULL	0	*afl mode*
NOT NULL	*any other value*	*afl++ mode*

is actually updated is allocated by the forkserver with a `malloc` call.[1]

- **afl mode**: In this case there is an actual SHM and a fuzzer that tracks the edge coverage.
- **afl++ mode**: Same as the previous point, but with a different communication protocol between the forkserver and the fuzzer.

The SO exports two different symbols:

- `uint8_t *__afl_area_ptr`: pointer to the SHM (or the local memory allocated with `malloc`)
- `void (*__forkserver_start_ptr)(void)`: pointer to the forkserver function that must be invoked by the main

These symbols are resolved inside the target binary in such a way that the binary has access to their values. Depending on the execution mode, these variables have different values.

- **standalone mode**:
  - *__forkserver_start_ptr*: The forkserver function will return without forking, because the target must be executed without tracing.
  - *__afl_area_ptr*: Allocated through malloc, because it does not need to be a shared map. In this case the edges are traced by the target, although this tracing is not actually used. However, for compatibility with the traced case, __afl_area_ptr is still used.
- **afl mode**:
  - *__forkserver_start_ptr*: In this case there is actual communication with the

[1] this fact is completely transparent to the target binary, which sees the relocated memory address as if it were a shared memory

fuzzer. The protocol used can be seen in algorithm 3.1.

- $__afl_area_ptr$: During the init function, the SHM address is associated with the pointer through a `shmat` function call, passing as parameter the ID of the SHM obtained from the environment variable.

- **afl++ mode**: This mode works in the same way as the previous one, but uses a different communication protocol. The communication protocol is shown precisely in fig. 1.12.

Algorithm 3.1 AFL forkserver algorithm

```
WRITE(FORKSRV_FD + 1, 0)
while true do
  READ(FORKSRV_FD)
  child_pid ← FORK()
  if child_pid = 0 then
    CLOSE(FORKSRV_FD) {The child must return from the forkserver and execute the main, so the child process can close the file descriptors}
    CLOSE(FORKSRV_FD + 1)
    return
  else
    WRITE(FORKSRV_FD + 1, child_pid) {Communicate the PID of the child to the fuzzer}
    status ← WAITPID(child_pid)
    WRITE(FORKSRV_FD + 1, status)
  end if
end while
```

libforkserver.so also has a destructor function called `void afl_forkserver_cleanup(void)`. A function declared as a destructor is a function with the attribute `__attribute__((destructor))`. A reference to a function declared with the previous attribute is placed inside *.fini_array* and is executed after the main. This function is used to detach the SHM from the current process (in the case of afl/afl++ execution mode), or to free the memory in the case of standalone execution mode.

3.2. Instrumentator Specification

In this case the discussion will vert on the **instrument.py** file. This is the file capable of actually instrumenting the target binary. It uses two libraries:

- **lief** [20]: a python library used mainly to add the dynamic dependency, the new sections and the relocation entries. This library is used for the Binary Modification phase.

- **GRAPE**: a binary rewriting framework used to instrument every IP found. This framework is used in the Instrumentation Phase.

3.2.1. Binary Modification

Before actually instrumenting the binary, the previously mentioned modifications must be made. The principal steps are summarized in fig. 3.1.

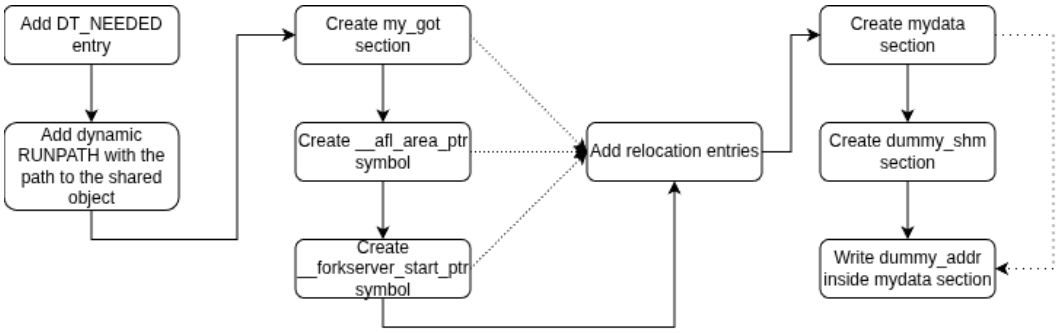


Figure 3.1: Binary modification phase in which the needed sections, relocations and dynamic dependencies are added. (The dotted arrows specify which information are needed to following steps)

create .my_got section

The created section has the following properties:

- *section type* = *PROGBITS*. This means that the section data is actually stored in the ELF file. For example, the *.bss* section is declared as NOBITS because its data is not stored in the ELF file, but a memory region must be allocated when the program starts.
- *section flags*
  - ALLOC: the section must occupy actual space in memory. This memory must be allocated during ELF loading.
  - WRITE: this section must be writable by the process.
- *section content*: this section must contain the resolved addresses of the two symbols that are relocated. So the content of this section is composed of 16 bytes (in 64-bit addressing, each resolved symbol occupies 8 bytes). The section content is automatically initialized to 0.

- *section alignment = 8.* Since this section will store 8-byte addresses, the alignment is set to 8.

This section, as stated before, must actually be loaded into memory. But this means that the section must be associated with a segment having the LOAD flag, with the same permissions as the section. **lief**, if specified, automatically adds a new segment for each added section.

create symbols

Two symbols must be created. These two symbols correspond to the two exported symbols of the SO, and so their names must match. Specifically, these symbols are created with:

- *symbol name*: since the symbols' names must match, the two symbol names are
  - *__afl_area_ptr*
  - *__forkserver_start_ptr*
- *symbol type = OBJECT.* The symbol represents a variable, not a function.
- *symbol binding = GLOBAL.* The symbol can be referenced by the dynamic linker. As stated in section 1.3, during ELF loading, the dynamic linker resolves all the symbols for which there is a relocation entry. A symbol can be exported or imported only if its binding is set to GLOBAL. There are LOCAL symbols that, by definition, are local to the object that defines them and therefore cannot be exported (and, consequently, cannot be imported either).
- *symbol visibility = DEFAULT.* The symbol has the default visibility of ELF. There are other types (such as HIDDEN or PROTECTED) that are used for symbols that are not generally visible outside the file.
- *symbol value = 0.* The symbol does not have a value. This indicates that this symbol must be resolved by the dynamic linker.
- *symbol size = 8.* The symbol size is 8 bytes.

Add relocation entries

Two relocation entries must be added. The first is used to resolve *__afl_area_ptr*, and the second to resolve *__forkserver_start_ptr*. In order to create a relocation entry, the following must be specified:

- *target address of relocation*: where the resolved value will be written (*.my_got* entry)

- *relocation type*: for both symbols, X86_64_GLOB_DAT is used (the usual type when dealing with GOT resolution)
- *relocation encoding*: for both symbols, RELA encoding is used, which consists in explicitly specifying the addend
- *relocation purpose*: for both symbols, a DYNAMIC purpose is used, because the symbol must be resolved at runtime by the dynamic linker
- *addend = 0*
- *relocation symbol*: the symbol to be resolved

The difference between the two relocation entries lies in the target address and in the relocation symbol. For *__afl_area_ptr*, the target address is the virtual address of the *.my_got* section.
For *__forkserver_start_ptr*, the target address is the virtual address of the *.my_got* section + 8.

Create .mydata section

All the considerations made for the previous section are still valid, apart from the section content. In fact, before actually running a target, AFL checks whether the string "__AFL_SHM_ID" is contained somewhere in the ELF file, in order to know if the target has been instrumented. If it is not found, AFL stops.[2] For this reason the aforementioned string is placed inside this section. The string will later be overwritten by other data, but it will pass the AFL check during the fuzzing setup. According to my implementation, this section will contain different pieces of information at different addresses:

- *.mydata+0x10*: contains the ID of the previous BB
- *.mydata+0x20*: contains the address of *dummy_shm* and after the SHM initialized by the initial instrumentation
- *.mydata+0x30*: contains the saved `rax` register, to preserve its value before and after the instrumentation
- *.mydata+0x40*: contains the saved RFLAGS, used to preserve the binary behavior before and after the instrumentation

Create .dummy_shm and write its address inside .mydata

[2]This check can actually be avoided by setting the environment variable AFL_SKIP_BIN_CHECK=1, as must be done when using ZAFL.

The section's type and flags are the same as the previous discussed section. The alignment has been set to 0x1000. The content of this section is all initialize to 0 with a size of 0x10000. This is done because the SHM dimension is 65536=0x10000 as previously stated in section 1.6.
As already seen in section 2.6 the address of the *.dummy_shm* must be written in *.mydata+0x20*. In the general case, current systems uses ASLR and this make impossible to know the actual address in which *.dummy_shm* will be loaded. So a relocation must be added. This relocation is different from the previous because there is not an explicit symbol to be resolved.

- *target address of relocation*: where the resolved value will be written (*.mydata+0x20*)
- *relocation type*: X86_64_RELATIVE. This is a relocation relative to the base address in which the binary will be loaded and does not require the resolution of a symbol. Explicitly the formulae used by the loader is $*P = B + A$ where P is the *target address*, B is the *Base Address*, A is the *addend*. So will write inside P the result of the address in which the binary will be loaded plus the addend
- *relocation encoding*: RELA encoding is used, which consists in explicitly specifying the addend
- *relocation purpose*: DYNAMIC
- *addend = .dummy_shm address*. Since the idea is to write the rebased virtual address of the *.dummy_shm* the addend must be the non rebased address

At the end of the **Binary Modification phase**, the modified binary is saved to the filesystem.

3.2.2. Instrumentation

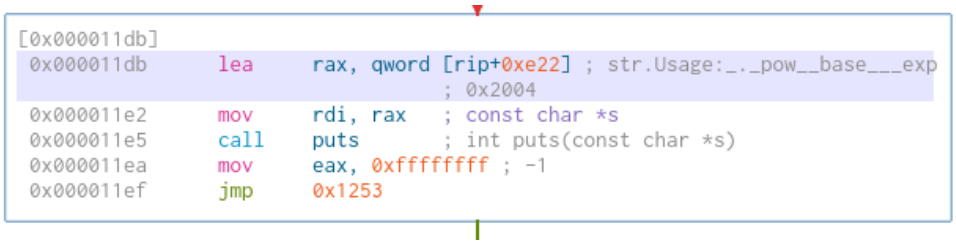
The instrumentation is done entirely using the GRAPE framework. It takes care of adjusting all the reference shifts caused by the altered *.text* section, as well as the eventual resizing of the section (caused by it exceeding its size due to the added instructions), and many other features. GRAPE first reconstructs the CFG of the program and then performs operations on the obtained CFG. The classes that the developed tool will use are:

- AddInstructionChange: used to add an instruction at a specific address of the binary CFG. By default, the instruction is added at the specified address, shifting the instruction that was previously there forward.

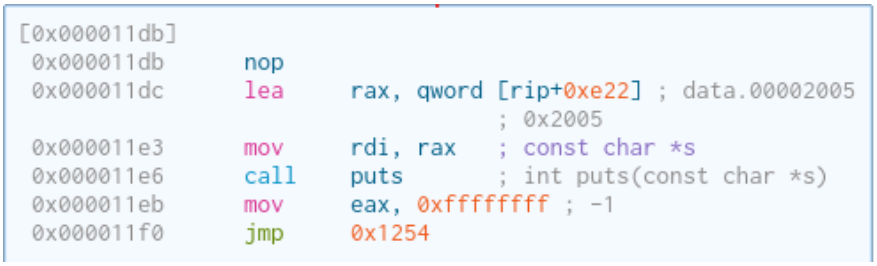
- RemoveInstructionChange: used to remove an instruction from a specific address of the binary.

These two classes take care of CFG modification (splitting a BB, merging two BBs, ...) but do not take care of RIP-relative instruction modification, together with symbol coherence. An example of an error caused by not taking care of modifying the RIP offset of single instructions is shown in fig. 3.2.

[3] In fig. 3.2 it can be seen that, after having inserted a `nop` instruction, the `lea` that



(a) Example of a Basic Block before the addition of an instruction



(b) Same Basic Block of fig. 3.2a with a nop instruction added

Figure 3.2: Example of the error caused by not taking care of RIP-relative instruction offsets

was at address 0x11db now shifts to address 0x11dc. For this reason the referenced location is no longer a valid data location. Before the addition, the computation was $0x11db + 0xe22 + 7 = 0x2004 < usage_string_location >$[4]. Now the computation is $0x11dc + 0xe22 + 7 = 0x2005 <?? >$

For these reasons, the last step of binary rewriting (after actually adding all the instructions to the BB) uses two other classes:

- RIPDataCoherenceAnalysis: adjusts the RIP-relative instructions.
- SymbolCoherenceAnalysis: adjusts symbols. For example, after inserting some instructions, the target address of another instruction may have shifted. The re-

[3] As can be seen, the jmp at 0x11ef (and after 0x11f0) seems to be shifted, but in reality the jmp instruction is always RIP-relative and the disassembler used [28] automatically performs the relative address resolution.

[4] The +7 is due to the instruction size

construction of the correct target address is performed by the SymbolCoherence phase.

Since GRAPE takes care of all the shifting and adjusting, the instrumentation algorithm only needs to take care of computing the right offset of the RIP-relative instructions to be instrumented and inserting the instructions.

Since the instructions must be added at a specific address, all the offsets are computed as $final_address - instrumentation_address - instruction_size$. For example, with reference to Listing 3.1, supposing the instrumentation address is 0x4100000c, offset_to_previous_saved_ID can be computed as follows:
$offset_to_previous_saved_ID = (.mydata.virtual_addr + 0x10) - 0x4100000c - 9$[5].

In algorithm 3.2 the initial instrumentation algorithm can be seen. As stated before, GRAPE automatically increases the size of the *.text* section if, after adding instructions, the section becomes too small to contain them all. When this happens, the *.mydata* and *.my_got* sections could be shifted forward, and the previously computed offsets for the RIP-relative instructions are no longer valid. For this reason, if a shift is detected, all the instructions are recomputed. Moreover, it can be seen that the instructions are inserted in reverse order (the loop index starts from the last instruction). This is because the instructions are inserted one on top of the other at the same address.

Initial Instrumentation

Listing 3.1 performs three main tasks:

- Finds the address of the forkserver function by dereferencing the resolved symbol, stores it in `rax`, and then performs an indirect call.
- Writes a random integer (between 0 and 65535[6]) as the previous BB ID.
- Dereferences the address of the SHM resolved by the dynamic linker and writes it into the *.mydata* section, in order to avoid dereferencing it every single time.

```
1 push rsi
2 push rdi
3 mov rax, QWORD PTR[rip + offset_to_relocated_forkserver_function]
4 mov rax, [rax]
5 call rax
6 pop rdi
```

[5]The -9 at the end is needed because the instruction at line 6 of Listing 3.1 uses 9 bytes
[6]Since the AFL shared map size is 65536, each Basic Block ID must be between 0 and 65535

Algorithm 3.2 Initial Instrumentation algorithm

Require: $addr, mydata_addr, mygot_addr$
Ensure: all the initial instructions are inserted in the correct order at address "addr"

1: $instructions \leftarrow$ COMPUTEINITIALINSTRUMENTATION$(addr, mydata_addr, mygot_addr)$ {COMPUTEINITIALINSTRUMENTATION computes the right offsets and returns a list of assembly instructions written as strings}
2: **for** $i =$ LEN$(instructions)$ **to** 1 **do**
3: $instr \leftarrow$ ASM$(instructions[i])$ {the ASM function converts the assembly instruction, passed as a string parameter, into machine code}
4: ADDINSTRUCTIONCHANGE$(addr, instr)$.MODIFY$(binary.cfg)$
5: **if** .mydata section shifted after .text enlargement **then**
6: $mydata_addr \leftarrow mydata.virtual_address$
7: $mygot_addr \leftarrow mygot.virtual_address$
8: $instructions \leftarrow$ COMPUTEINITIALINSTRUMENTATION$(addr, mydata_addr, mygot_addr)$ {recompute all the instructions with the new offsets. GRAPE will take care of shifting the instructions already added}
9: $instr \leftarrow$ ASM$(instructions[i])$
10: REMOVEINSTRUCTIONCHANGE$(addr)$.MODIFY$(binary.cfg)$ {remove and reinsert the last instruction, to make sure that the offset to the .mydata section is correct}
11: ADDINSTRUCTIONCHANGE$(addr, instr)$.MODIFY$(binary.cfg)$
12: **end if**
13: **end for**

```
pop rsi
mov WORD PTR[rip + offset_to_previous_saved_ID], random_int
mov rax, [rip + offset_to_relocated__afl_area_ptr]
mov rax, QWORD PTR [rax]
mov QWORD PTR[rip + offset_to_saved_shared_memory_address], rax
```

Listing 3.1: Initial Instrumentation

Basic Block Instrumentation

- Save rax and RFLAGS
- Perform SHM update
- Restore RFLAGS and rax

The instrumentation added for each BB can be seen in Listing 3.2.

```
mov QWORD PTR[rip + offset_to_saved_rax], rax
lahf,
seto al
mov WORD PTR [rip + offset_to_saved_RFLAGS], ax
movzx rax, WORD PTR[rip + offset_to_previous_saved_ID]
xor ax, random_current_basic_block_ID
add rax, QWORD PTR[rip + offset_to_saved_shared_memory_address]
```

```
8  inc BYTE PTR[rax]
9  mov WORD PTR[rip + offset_to_precedent_bb_saved],
       right_shifted_current_ID
10 mov ax, WORD PTR [rip + offset_to_saved_RFLAGS]
11 add al, 0x7f
12 sahf
13 mov rax, QWORD PTR[rip + offset_to_saved_rax]
```

Listing 3.2: Basic Block Instrumentation

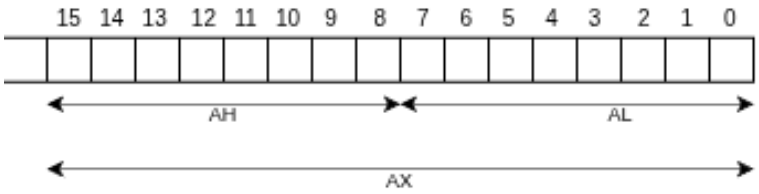
In Listing 3.2, all the considerations previously made for the offset computation are still valid. random_current_basic_block_ID and right_shifted_current_ID are computed by the instrumenter and hardcoded into each BB.

A different consideration must be made in order to understand how to save and restore RFLAGS. The x86-64 mechanism to store RFLAGS is the instruction `pushfq` that store on the stack the entire RFLAGS, but, as already stated in section 2.4, is not possible in the general case to use the stack, because some functions uses the **red zone** mechanism to store local variable and there is the risk of overwrite it. For this reason a different approach must be used. The instruction `lahf` (*Load AH from FLAGS*) copy the first 8 bits of RFLAGS inside the ah register. The problem is that this instruction does not capture the Overflow flag. This is done through `seto al`, thats copy the OF into the register al. The complete procedure could be seed in fig. 3.3. The value of ax is then written into *.mydata+0x40*.
A different consideration must be made for restore RFLAGS. First of all the value previously stored into *.mydata+0x40* is copied into ax register. Then the instruction `sahf` (*Store AH to FLAGS*) copy back the previously saved values of flags from ah register to the first 8 bit of RFLAGS. The problem is that does not exist an instruction that copy back the value of OF. The solution is the instruction at line 11 of Listing 3.2.

- If the saved value of OF is 0 $al + 0x7f = 0x7f$ and the actual OF is set to 0
- If the saved value of OF is 1 $al + 0x7f = 0x00$ and the actual OF is set to 1

As can be seen in both the case the value of OF is restored correctly.

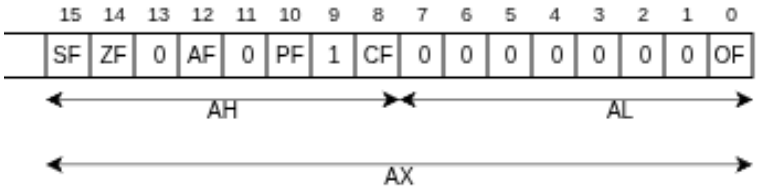
For the BB instrumentation, the algorithm must also take care of address shifting during instrumentation. Let's consider the example of fig. 2.3 and the BB addresses found in that section. These addresses were $[0x11a7, 0x11b5, 0x11bd, 0x11db, 0x11f1, 0x1253]$. The instrumentation seen in Listing 3.2 occupies a total of 67 bytes. Supposing the location 0x11bd has been instrumented, all the following locations must then be shifted by 67 bytes. And this computation must be done for each IP found.



(a) ax register before `lahf`



(b) ax register after `lahf`



(c) ax register after `seto al`

Figure 3.3: Example ax register during first instructions of the basic block instrumentation

But this approach is quite impractical. GRAPE, in order to overcome the issue of changing addresses, uses *labels*. It associates a unique label to each BB. The BB can be uniquely retrieved using this label. If the BB changes its starting address, the label will not change. So, during the IP search, instead of keeping the addresses of the basic blocks saved, the tool keeps the labels of the BBs to be instrumented saved, and only afterwards retrieves the addresses of these BBs. The final approach can be seen below.

After the instrumentation of each BB according to algorithm 3.3, the tool performs RIPDataCoherenceAnalysis and SymbolCoherenceAnalysis, adjusting all the shifted references, and overwrites the file that **lief** had previously written.

Finally, the tool must check that the `e_phoff` field of the ELF header coincides with the actual offset of the **Program Header Table**, and thus with the `p_offset` field of the **PT_PHDR** segment.
In some binaries, adding new segments (for *.mydata* and *.my_got*) and adding instructions to other segments can cause the **Program Header Table** to shift, and the `e_phoff` field

Algorithm 3.3 Instrument Basic Block algorithm

Require: $bbs = [bb_label_1 \dots bb_label_n], mydata_addr, mygot_addr$
Ensure: all the Basic Blocks are instrumented correctly

```
1:  for bb_label ∈ bbs do
2:      bb ← binary.getBasicBlockFromLabel(bb_label) {"binary" is the main object
        used by GRAPE and is a virtual representation of the ELF it is rewriting}
3:      addr ← bb.initial_address
4:      instructions ← computeBasicBlockInstrumentation(addr, mydata_addr, mygot_addr)
5:      for i = len(instructions) to 1 do
6:          instr ← asm(instructions[i])
7:          AddInstructionChange(addr, instr).modify(binary.cfg)
8:          if .mydata section shifted after .text enlargement then
9:              mydata_addr ← mydata.virtual_address
10:             mygot_addr ← mygot.virtual_address
11:             instructions ← computeBasicBlockInstrumentation(addr, mydata_addr, mygot_addr)

12:             RemoveInstructionChange(addr).modify(binary.cfg)
13:             AddInstructionChange(addr, instr).modify(binary.cfg)
14:         end if
15:     end for
16: end for
```

not to be updated correctly.

The tool first reads the ELF header using the **lief** library, obtaining `e_phoff`, and then cycles through all the binary segments until it finds the **PT_PHDR** segment. When it finds the segment, it checks whether the two fields coincide. If not, it manually rewrites the 8 bytes (for 64 bit programs) that represents `e_phoff` (the 0x20th byte of the header), adjusting it to the correct value. The "manual" rewriting is performed without using an actual library, assuming little-endian addressing. This is done in order to avoid libraries (such as **lief**) adjusting other fields.

4 | Experiments

This chapter presents the objectives of this thesis, the dataset used to verify whether these objectives are achieved, the experimental setup in which the tests are executed, and the tests performed, together with their results.

4.1. Goals

In order to prove that the instrumentation approach is valid, I have chosen three main goals:

- **Goal I** : The instrumented binary must be compatible with the original one. This means that the behavior of the program must be preserved after the instrumentation.
- **Goal II** : The instrumented binary must be compatible with AFL++ communication protocol.
- **Goal III** : Show that the instrumentation overhead is competitive with other binary-only rewriters and lower than that of dynamic binary translators.

In order to prove that the tool satisfies these goals, the following experiments were conducted:

- **Experiment I** : Test the original binary and the instrumented binary with the same input, without using a fuzzer, in order to prove that the behavior of the program is preserved after the instrumentation (Goal I).
- **Experiment II** : Perform a fuzzing campaign with three different instrumentations:
  - AFL++ in QEMU mode
  - ZAFL
  - Mine

  The chosen fuzzer is AFL++. This test aims to prove Goal II and Goal III.

4.2. Datasets and Experiment Setup

Experiment I

The experiment consists in feeding the same input to both the original binary and the modified one, and checking whether the outputs coincide. The dataset is taken from the *real-world software* dataset used in the ZAFL paper [26]. Not all the software was used. Table 4.1 lists the binaries used, with their versions. All the binaries considered for this

Table 4.1: Binaries extracted from ZAFL *real-world software* dataset

Binary name	Version
bsdtar	bsdtar 3.7.2 - libarchive 3.7.2
readelf	GNU Binutils 2.42
sfconvert	0.3.6
tcpdump	4.99.4
unrtf	0.21.10

experiment are stripped.

Experiment II

The dataset used for the second experiment is LAVA-M [10], in the same version used by ZAFL [26]. This dataset is composed of four coreutils binaries (*base64* , *md5sum* , *uniq* , *who*), to which synthetic bugs were added with the LAVA framework. Each bug has a unique ID and is protected by a checksum. An input can trigger a synthetic bug only if random mutation manages to satisfy the checksum. When a specific bug is triggered, the unique ID of that bug is printed and the program then crashes intentionally.
The coreutils version used is **8.24**. As reported by Chen et al. [8], the LAVA-M version of *md5sum* was modified incorrectly: when run with the `-c` option, it crashes on every input. This affects our second experiment, in which the fuzzing seed must be parsed with `-c`. Since every input triggers a crash, the results for this binary are not meaningful. The experiment was run with AFL++ version 5.03c, over a 24-hour fuzzing campaign with three repetitions for each tested binary. The experiment was executed inside a KVM virtual machine (full virtualization) exposing 16 virtual CPUs, hosting Ubuntu 24 as guest operating system, with 32 GB of RAM. All the different runs was executed concurrently; each was pinned to a dedicated virtual CPU. The 3 different instrumentation methods are tested indipendently; parallely are executed the 3 runs of one of the instrumentation method on each of the 4 binaries, having a total of 12 virtual cores occupied. The seeds for each runs are different. The initial corpus is composed of only 1 test case.
The general AFL++ command used to each fuzzing campaign is:

`afl-fuzz -i <input_dir> -o <findings_dir> -V 86400 -- <binary>` ; with some difference depending on the binary that is being tested and the instrumentation method:

- For the binaries instrumented with ZAFL the environment variable "AFL_SKIP_BIN_CHECK=1" must be set
- For the binaries fuzzed with AFL++ in QEMU mode, the option "-Q" must be added to the command

Moreover the 4 binaries are executed in different ways:

- *base64* : `base64 -d`
- *md5sum* : `md5sum -c @@`
- *uniq* : `uniq`
- *who* : `who @@`

4.3. Experiment I

Among the binaries listed in table 4.1, *tcpdump* and *unrtf* could not be instrumented: the failure occurs during the GRAPE CFG reconstruction (specifically during `Angr` CFG analysis).
Each binary is executed different times, with different inputs and different execution options. The output of the original and the instrumented are then compared to check if are the same or not. The results of the experiment are shown in table 4.2.

Table 4.2: Results of experiment I

	Instrumented	total tested inputs	pass	fail
bsdtar	✓	34	34	0
readelf	✓	54	4	50
sfconvert	✓	52	52	0
tcpdump	✗	–	–	–
unrtf	✗	–	–	–

With regard to *readelf*, the instrumented binary encounters a `SIGSEGV` error in all the failed tests. This is due to the fact that, when new sections and segments are added to the binary, the **lief** library automatically reorganizes the ELF layout, causing some sections (e.g. *.bss*) to be shifted. The instructions that referenced these sections will, after the layout change, reference wrong addresses, causing a `SIGSEGV` when they try to

access them.
The 4 test passed by readelf are 4 inputs that in the original binary causes the execution to stop with an exit code different from 0. These inputs does not cause a `SIGSEGV` in the original as in the instrumented file, but "pass" the test because in both case the exit code is different from 0. A possible reason for the difficulty encountered in instrumenting this dataset could be the fact that this binaries are stripped. The absence of symbols could create more problems in CFG reconstruction.

4.4. Experiment II

Among the four LAVA-M binaries, none fails during the instrumentation phase. However, two of them (i.e. *md5sum* and *who*) result in an incorrect instrumentation: *md5sum* was already corrupted, as stated before, while *who* was incorrectly instrumented by the tool. The results of the experiment are shown in table 4.3. The AFL++ in QEMU mode run and the ZAFL run of the `md5sum` and `who` are not shown in table 4.3 because is impossible to compare those runs with the tool discussed in this thesis. Table 4.3 is therefore divided into two subtables, one per fuzzed binary. Each table reports some parameters obtained for each of the three runs performed.
As can be seen, the tool developed in this thesis finds a considerably smaller number of edges than the other tested methods. This could be caused by a different edge instrumentation method (for example, because the tool does not instrument function calls).
Since each binary has been injected with LAVA synthetic bugs, the *"uniq_ bugs"* contains the number of uniq bugs triggered by the run. The id of each uniq bug are reported in table 4.4. The number of uniq bugs triggered is always less or equal to the number of uniq crashes because different crashes could trigger the same bug.

ZAFL is the only tool capable of executing *md5sum* with the `-c` option but the high number of crashes if given by errors of the binary. Infact none of the found uniq crashes actually trigger a LAVA Bug, but trigger SIGSEGV errors that make the entire statistic of md5sum unreliable.

Table 4.5 shows the size of the instrumented files in MBytes and the average executions per second obtained from table 4.3. As already discussed, the tool developed in this thesis produces larger binaries, because the SHM update is instrumented inline. As shown in table 4.5 the size of all the instrumented binaries is similar regardless of how many locations have been instrumented. This could be a sign of a fixed dimension incremention given by the added section. The average execution speed is only slightly higher than that of ZAFL. Since the tool execution speed, as can be seen in table 4.3, is not always higher

with respect to the ZAFL execution speed, and only slightly higher in average, it is not possible to conclude an actual improvement in the execution speed.
Moreover is present the speedup with respect to QEMU mode. The speedup is obtained dividing the average execution per second obtained with mine instrumentation and ZAFL by the average executions per second obtanied with AFL++ in QEMU mode.

Worth mention that ZAFL perform an optimization of the Intermediate Representation, that surely speed up the instrumented binary execution. With inline instrumentation is possible to reach performance that are comparable to the one obtained by ZAFL, without an actual optimization.

(a) `base64`

	AFL++ QEMU mode			mine			ZAFL		
	run1	run2	run3	run1	run2	run3	run1	run2	run3
execs/s	398	445	352	1706	1860	1776	1577	1792	1749
edges_found	690	690	690	362	386	362	640	672	672
corpus_count	162	169	178	151	214	149	143	205	201
stability	100%	100%	100%	100%	100%	100%	100%	100%	100%
saved_crashes	7	3	2	5	102	13	10	115	118
uniq_bugs	3	3	2	5	46	7	7	48	48

(b) `uniq`

	AFL++ QEMU mode			mine			ZAFL		
	run1	run2	run3	run1	run2	run3	run1	run2	run3
execs/s	568	577	577	1751	1713	1783	1774	1560	1606
edges_found	473	473	473	242	242	242	443	443	443
corpus_count	97	101	100	86	88	87	90	89	85
stability	100%	100%	100%	100%	100%	100%	100%	100%	100%
saved_crashes	2	0	0	0	0	0	0	1	0
uniq_bugs	0	0	0	0	0	0	0	1	0

Table 4.3: Fuzzing statistics over a 24-hour fuzzing campaign on the LAVA-M dataset using AFL++ in QEMU mode, ZAFL instrumentation + AFL++ fuzzer, and my instrumentation + AFL++ fuzzer

The results for `md5sum` and `who` are not presented here because the comparison with the tool discussed in this thesis is not possible

(a) LAVA-M uniq bugs triggered by the runs in `base64`

run name	uniq bugs IDs
afl1	['784', '792', '804']
afl2	['276', '792', '804']
afl3	['276', '790']
mine1	['784', '788', '792', '804', '815']
mine2	[798', '804', '805', '806', '815', '817', '831', '835', '841', '842', '843']
mine3	['778', '782', '784', '788', '792', '804', '817']
zafl1	['253', '276', '560', '774', '778', '782', '792']
zafl2	['1', '222', '235', '253', '255', '274', '276', '278', '284', '386', '521', '526', '527', '554', '556', '558', '560', '562', '566', '572', '573', '576', '582', '583', '584', '774', '776', '778', '780', '782', '784', '786', '788', '790', '792', '798', '804', '805', '806', '813', '815', '817', '831', '832', '835', '841', '842', '843']
zafl3	['1', '222', '235', '253', '255', '274', '276', '278', '284', '386', '521', '526', '527', '554', '556', '558', '560', '562', '566', '572', '573', '576', '582', '583', '584', '774', '776', '778', '780', '782', '784', '786', '788', '790', '792', '798', '804', '805', '806', '813', '815', '817', '831', '832', '835', '841', '842', '843']

(b) LAVA-M uniq bugs triggered by the runs in `uniq`

run name	uniq bugs IDs
afl1	[]
afl2	[]
afl3	[]
mine1	[]
mine2	[]
mine3	[]
zafl1	[]
zafl2	[472]
zafl3	[]

Table 4.4: LAVA-M uniq bugs triggered by the runs performed in table 4.3

		file_size[MByte]	avg execs/sec	speedup w.r.t. AFL++ in QEMU mode
base64	mine	8,32	1781	4,47
	zafl	0,14	1706	4,28
md5sum	mine	8,32	✗	–
	zafl	0,15	515	–
uniq	mine	8,32	1749	3,04
	zafl	0,15	1647	2,87
who	mine	10,78	✗	–
	zafl	1,72	456	–

Table 4.5: Instrumented binaries statistics

Legend: The "–" symbol is placed on that binaries where the comparison can not be done

5 | Limitations and Future Works

As demonstrated by the tests performed and discussed in the previous chapter, the current implementation has some limitations. In this chapter, I will discuss the main limitations of the current implementation and try to outline a direction for future work in this field.

5.1. Limitations

The main limitation is that the developed tool fails to correctly modify some binaries, as shown in table 4.2. I have identified some possible causes of instrumentation failure. One of these occurs in some binaries, where the addition of segments (needed to hold the new sections) causes other sections in other segments to shift. In fact, according to the **LIEF** documentation, LIEF does not guarantee preservation of the ELF layout. For instance, when the dynamic dependency is added to the binary, some sections are modified. Specifically:

- *.dynstr*: the string 'libforkserver.so' must be added to the dynamic string section.
- *.dynamic*: LIEF adds a DT_NEEDED entry that points to the newly added string in *.dynstr*.

If, during either of these two additions, LIEF exceeds the available space in the segment containing the grown section, it reorganizes the segments. This causes a complete displacement of the data sections (such as *.bss, .data, .rodata*, ...) without a corresponding fix to the instructions that reference these data sections. This can cause a segmentation fault when, for example, an instruction tries to access a value stored in *.bss*. In both PIE and non-PIE binaries, the non-linear displacement between the *.bss* section's address before and after the binary modification causes instructions to no longer point to the correct address. This fact can be seen as the underlying problem both for *readelf* and *sfconvert* of table 4.2. In *who* of table 4.3 instead, the problem needs to be traced in some error inside the instrumentation that cause the disalignment of the instructions of a BB, causing some of these instruction to be illegal instructions.
Moreover, for other binaries, the problem arise in the actual instrumentation phase, high-

lighting a certain fragility of the tool to instrument any type of binary.

Another important limitation that must be mentioned is that the presented approach is not thread-safe. Infact, the rax value is saved in a global memory area that is not thread safe. For this reason, if two parallel threads reach a SHM update block at the same time, the value inside of save rax register will be overwritten, causing one of the two threads to restore an incorrect value of rax. The same problem happens with previous BB ID saved. Supposing that, at a certain point, four different threads are active, each thread will have, in general, a different previous BB ID.

In case of dynamic linking, where the code of external libraries is not actually inside the binary, the library functions cannot be instrumented, because the tool is capable of instrumenting only the code sections inside the binary that are reachable from main.

Another limitation worth mentioning is that the developed tool works only with ELF files, lacking compatibility with the Windows PE executable format.

5.2. Future Works

The main work to be done in the future consists in making this type of solution more resilient to binary corruption. The approach of instrumenting the SHM update directly inline within each BB reach performance comparable with the current state-of-the-art. In particular ZAFL, as stated in section 1.6, perform an optimization to the Intermediate Representation. The approach discussed in this thesis reach similar performance, without an actual optimization. For this reason, a possible direction for future work could be to apply current state-of-the-art techniques to this approach. For example, using the SHM update rule presented in this thesis, several hash collisions could occur in the SHM, causing the fuzzer to misinterpret coverage, as demonstrated by Shuitao Gan et al. [14]. In fact, that paper presents a new approach to SHM updates, which was later added as a supported instrumentation method in the AFL++ framework. Implementing a collision-free coverage algorithm as SHM update rule could be a possible improvement over the proposed approach.

Furthermore, implementing a sanitization mechanism could be another research direction building on the work done in this thesis, for example by implementing an Address Sanitizer [30] or a Thread Sanitizer [29].

6 | Conclusion

This work had investigated the possibility to implement an inline edge coverage instrumentation for binary only grey-box fuzzing that uses GRAPE binary rewriter framework to add instructions to the binaries. The instrumentation added is compatible with AFL and AFL++ and require no access to the source code. The following chapter summarize the work discussed in the previous chapters, asseses to which of the goals in section 4.1 were achieved and close with some final remarks. The limitations of the vurrent implementation have been already discussed in chapter 5 and so are not repeated here.

6.1. Summary of the work

Coverage-guided grey-box fuzzing use the coverage feedback obtained through instructions instrumented inside the binary. Dynamic approached such as AFL++ in QEMU or FRIDA mode, DynamoRIO ... insert the coverage tracing instruction dynamically within each BB. This approach is easy to deploy but costly in terms of overheads. Static approaches, such the one implemented in this work, statically rewrites the binary introducing the coverage tracing instruction, making the instrumented binary quite fast in term of overheads but with difficulty in maintain correctness.

The presented approach is based on the injection of a shared object (`libforkserver.so`) that implement the communication protocol with the fuzzer, implement the forkserver logic and attach the shared memory to the process. In order to inject the shared object a `DT_NEEDED` dependency is added to the binary, alongside with some relocation entries and some new sections, used as a support to the instrumentation. This has been implemented in the `instrument.py` file, togheter with the Instrument point search and the instrumntation. The binary modification and the actual instrumentation are made available thanks to two libraries. The binary modification (add dynamic dependency, relocations and sections) is done with LIEF python library. The actual instrumentation is made with the GRAPE binary rewriting framework.

6.2. Assessment of the goals

Goal I - behavior preservation

The Experiment I was performed on five stripped binaries taken from the ZAFL dataset. Among these, only two of them (`bsdtar` and `sfconvert`) preserve their behavior on all the tested input (34 out of 34 for `bsdtar` and 52 out of 52 for `sfconvert`). `readelf` was instrumented but fails to maintain the behavior. The cause has been found to be the ELF layout modification made automatically by LIEF that change the address in which some segments are loaded without changin the instructions that reference these segments. This cause a segmentation fault each time an instruction try to access the old memory address. `readelf` pass 4 out of 54. The result is given by the fact that also the original binary terminate with a non-zero exit code and the behavior of the original and the instrumneted is erroneously considered to be the same. Moreove, Experiment II reveal one additional failure on `who`, caused by an error on the instrumentation itself that still has to be investigated. For all these reason, Goal I is considered only partially achieved. Infact the robustness of the pipeline presented in not sufficiently strong to address any binary in the general case

Goal II - compatibility with AFL

Experiment II shown that forkserver injected into the binary can correctly communicate with AFL++ over 24-hours fuzzing campaign. Both the forkserver handshake, the shared memory update and the collection of the exit status of the target works correctly in each repetition on `base64` and `uniq`.

Goal III - overhead with respect to other binary-only approaches.

On the two correctly instrumented LAVA-M binaries (`base64` and `uniq`), the version instrumneted with the tool presented in this thesis sustain on average about 1760 and 1670 executions per second respectively, which is between 2.9 and 4.4 times the throughput of AFL++ in QEMU mode (which has about 400 and 570 executions per second rispectively). With respect to ZAFL instrumnetation method, the throughput is of the same order and the difference is smaller than the variation observed between repetitions of the same configuration. So neither an improvement nor a deterioration of the perfomrance could be claimed. Two caveat must be kept in mind while these results are read : first of all the number of edges that, in the case of the tool discussed in this work is considerably smaller with respect to AFL++ in QEMU mode and ZAFL. I had suppose that this is due to the choice to not instrument the calls, reducing the number of edges of the CFG that are actually been instrumented. The second is that the evaluation done rely

on small number of binaries and does not include a baseline for compile-time instrumentation, so the overheads could not be quantified in absolute terms. Therefore Goal III is achieved with respect to Dynamic Binary Instrumentation. For the competitivity with other Static Binary Instrumentation methods the Goal III remains partially achieved. Infact the achieved performance are comparable with ZAFL but the fact that this last tool preserve the behavior in more casesmake the developed tool not competitive for the moment with the state-of-the-art.

6.3. Final remarks

In the end I have showed that an inline, stack-free edge coverage instrumentation build on GRAPE is feasible and the obtained executions per second in a fuzzing campaign are well above DBI and in average comparable with the throughput of other SBI. The experiments shows that the approach is currently weak. Infact, although most of the failures does not originate from the instrumentation logic itself but from the infrastructure around it, the modification of a binary adding relocations, sections and dependencies without modify ELF layout remain a problem with the proposed approach. This could be solved scanning the instructions searching the one that reference the old sections and update their offset, in order to reference the same location of the new section.

For all these reason, the developed tool should be consider a working prototype and a proof of concept of the approach, rather than a general porpouse rewriter. Its performance results are promising but preliminary given the small number of binary on which has been tested.

Bibliography

[1] AFL++ contributors. AFL++ Frida Mode. `https://github.com/AFLplusplus/AFLplusplus/tree/stable/frida_mode`, 2026. Accessed: 2026-09-13.

[2] AFL++ Developers. Fuzzing binary-only targets. URL `https://aflplus.plus/docs/fuzzing_binary-only_targets/`.

[3] D. Andriesse, X. Chen, V. van der Veen, A. Slowinska, and H. Bos. An in-depth analysis of disassembly on full-scale x86/x64 binaries. In *25th USENIX Security Symposium (USENIX Security 16)*, pages 583–600, Austin, TX, 2016. USENIX Association. ISBN 978-1-931971-32-4. URL `https://www.usenix.org/conference/usenixsecurity16/technical-sessions/presentation/andriesse`.

[4] D. Bruening, T. Garnett, and S. Amarasinghe. An infrastructure for adaptive dynamic optimization. In *International Symposium on Code Generation and Optimization, 2003. CGO 2003.*, pages 265–275, 2003. doi: 10.1109/CGO.2003.1191551.

[5] B. Buck and J. K. Hollingsworth. An api for runtime code patching. *The International Journal of High Performance Computing Applications*, 14(4):317–329, 2000. doi: 10.1177/109434200001400404. URL `https://doi.org/10.1177/109434200001400404`.

[6] M. Böhme, V.-T. Pham, and A. Roychoudhury. Coverage-based greybox fuzzing as markov chain. In *Proceedings of the 2016 ACM SIGSAC Conference on Computer and Communications Security*, CCS16, pages 1032–1043. ACM, Oct. 2016. doi: 10.1145/2976749.2978428.

[7] A. Charles, A. Herrera, P. Oslington, and A. Tiu. Pear: A static binary rewriting framework for binary-only fuzzing. June 2026. doi: 10.48550/ARXIV.2606.02126.

[8] P. Chen and H. Chen. Angora: Efficient fuzzing by principled search. In *2018 IEEE Symposium on Security and Privacy (SP)*, pages 711–725, 2018. doi: 10.1109/SP.2018.00046.

[9] S. Dinesh, N. Burow, D. Xu, and M. Payer. Retrowrite: Statically instrumenting

cots binaries for fuzzing and sanitization. In *2020 IEEE Symposium on Security and Privacy (SP)*, pages 1497–1511, 2020. doi: 10.1109/SP40000.2020.00009.

[10] B. Dolan-Gavitt, P. Hulin, E. Kirda, T. Leek, A. Mambretti, W. Robertson, F. Ulrich, and R. Whelan. Lava: Large-scale automated vulnerability addition. In *2016 IEEE Symposium on Security and Privacy (SP)*, pages 110–121, 2016. doi: 10.1109/SP.2016.15.

[11] G. J. Duck, X. Gao, and A. Roychoudhury. Binary rewriting without control flow recovery. In *Proceedings of the 41st ACM SIGPLAN Conference on Programming Language Design and Implementation*, PLDI 2020, page 151–163, New York, NY, USA, 2020. Association for Computing Machinery. ISBN 9781450376136. doi: 10.1145/3385412.3385972. URL `https://doi.org/10.1145/3385412.3385972`.

[12] A. Fioraldi, D. Maier, H. Eißfeldt, and M. Heuse. AFL++ : Combining incremental steps of fuzzing research. In *14th USENIX Workshop on Offensive Technologies (WOOT 20)*. USENIX Association, Aug. 2020. URL `https://www.usenix.org/conference/woot20/presentation/fioraldi`.

[13] A. Fioraldi, D. C. Maier, D. Zhang, and D. Balzarotti. Libafl: A framework to build modular and reusable fuzzers. In *Proceedings of the 2022 ACM SIGSAC Conference on Computer and Communications Security*, pages 1051–1065, 2022.

[14] S. Gan, C. Zhang, X. Qin, X. Tu, K. Li, Z. Pei, and Z. Chen. Collafl: Path sensitive fuzzing. In *2018 IEEE Symposium on Security and Privacy (SP)*, pages 679–696, 2018. doi: 10.1109/SP.2018.00040.

[15] X. Gao, G. J. Duck, and A. Roychoudhury. Scalable fuzzing of program binaries with e9afl. In *2021 36th IEEE/ACM International Conference on Automated Software Engineering (ASE)*, pages 1247–1251, 2021. doi: 10.1109/ASE51524.2021.9678913.

[16] P. Godefroid, M. Y. Levin, and D. Molnar. Sage: whitebox fuzzing for security testing. *Communications of the ACM*, 55(3):40–44, Mar. 2012. ISSN 1557-7317. doi: 10.1145/2093548.2093564.

[17] M. Heuse. Afl-dyninst, 2018. *URL: https://github. com/vanhauser-thc/afl-dyninst*.

[18] M. Heuse. AFL-DynamoRIO. `https://github.com/vanhauser-thc/afl-dynamorio`, 2018. Accessed: 2026-09-13.

[19] M. Heuse. AFL-PIN. `https://github.com/vanhauser-thc/afl-pin`, 2018. Accessed: 2026-09-13.

[20] LIEF. LIEF: Library to instrument executable formats. `https://github.com/lief-project/LIEF`. Accessed: 2026-10-12.

[21] LLVM Project. libfuzzer – a library for coverage-guided fuzz testing. `https://llvm.org/docs/LibFuzzer.html`. Accessed: 2026-09-11.

[22] C.-K. Luk, R. Cohn, R. Muth, H. Patil, A. Klauser, G. Lowney, S. Wallace, V. J. Reddi, and K. Hazelwood. Pin: building customized program analysis tools with dynamic instrumentation. *SIGPLAN Not.*, 40(6):190–200, June 2005. ISSN 0362-1340. doi: 10.1145/1064978.1065034. URL `https://doi.org/10.1145/1064978.1065034`.

[23] S. Mallissery and Y.-S. Wu. Demystify the fuzzing methods: A comprehensive survey. *ACM Computing Surveys*, 56(3):1–38, Oct. 2023. ISSN 1557-7341. doi: 10.1145/3623375.

[24] Marsman1996. Missing some instrumentations when instrumenting binary with AFL. `https://github.com/HexHive/retrowrite/issues/28`, Aug. 2021. GitHub Issue #28, HexHive/retrowrite repository. Accessed: 2026-09-13.

[25] B. P. Miller, L. Fredriksen, and B. So. An empirical study of the reliability of unix utilities. *Communications of the ACM*, 33(12):32–44, Dec. 1990. ISSN 1557-7317. doi: 10.1145/96267.96279.

[26] S. Nagy, A. Nguyen-Tuong, J. D. Hiser, J. W. Davidson, and M. Hicks. Breaking through binaries: Compiler-quality instrumentation for better binary-only fuzzing. In *30th USENIX Security Symposium (USENIX Security 21)*, pages 1683–1700. USENIX Association, Aug. 2021. ISBN 978-1-939133-24-3. URL `https://www.usenix.org/conference/usenixsecurity21/presentation/nagy`.

[27] O. A. V. Ravnås. Frida: Dynamic instrumentation toolkit for developers, reverse-engineers, and security researchers, 2026. URL `https://github.com/frida/frida`.

[28] RizinOrg. Cutter, 2026. URL `https://github.com/rizinorg/cutter`. Accessed September 27, 2026.

[29] J. Schilling, A. Wendler, P. Görz, N. Bars, M. Schloegel, and T. Holz. A binary-level thread sanitizer or why sanitizing on the binary level is hard. In *33rd USENIX Security Symposium (USENIX Security 24)*, pages 1903–1920, 2024.

[30] K. Serebryany, D. Bruening, A. Potapenko, and D. Vyukov. {AddressSanitizer}: A fast address sanity checker. In *2012 USENIX annual technical conference (USENIX ATC 12)*, pages 309–318, 2012.

[31] M. Wenzl, G. Merzdovnik, J. Ullrich, and E. Weippl. From hack to elaborate technique—a survey on binary rewriting. 52(3), 2019. ISSN 0360-0300. doi: 10.1145/3316415. URL https://doi.org/10.1145/3316415.

[32] J. Xiao, P. Jiang, Z. Zhao, R. Huang, J. Liu, and D. Li. Robust, efficient, and widely available greybox fuzzing for COTS binaries with system call pattern feedback. In *34th USENIX Security Symposium (USENIX Security 25)*, pages 6239–6258, Seattle, WA, Aug. 2025. USENIX Association. ISBN 978-1-939133-52-6. URL https://www.usenix.org/conference/usenixsecurity25/presentation/xiao-jifan.

[33] M. Zalewski. Technical "whitepaper" for afl-fuzz. https://github.com/google/AFL/blob/master/docs/technical_details.txt. Accessed: 2026-09-19.

[34] M. Zalewski. American Fuzzy Lop. http://lcamtuf.coredump.cx/afl/, 2017. Accessed: 2026-09-13.

[35] X. Zhu, S. Wen, S. Camtepe, and Y. Xiang. Fuzzing: A survey for roadmap. *ACM Computing Surveys*, 54(11s):1–36, Jan. 2022. ISSN 1557-7341. doi: 10.1145/3512345.

List of Acronyms

ASLR Address Space Layout Randomization. 39, 47

BB Basic Block. 2, 4, 5, 17–19, 23, 30–32, 34, 39, 46, 48–52, 63–65

CFG Control Flow Graph. 2, 4, 5, 19, 31, 47, 48, 57, 58, 66

DBI Dynamic Binary Instrumentation. 3, 4, 16–18, 22, 67

DFS Depth First Search. 30

ELF Executable and Linkable Format. 7, 8, 12, 44–46, 52, 53, 57, 63, 64, 66, 67

GOT Global Offset Table. 8, 9, 11, 13, 28–30, 35, 36, 39, 46

IP Instrumentation Point. 5, 6, 30–32, 34–36, 39, 44, 51, 52

IR Intermediate Representation. 6, 19

JIT Just in-time. 18

PIE Position Indipendent Executable. 39

PLT Procedure Linkage Table. 8, 9, 11

SBI Static Binary Instrumentation. 3, 16–18, 22, 67

SHM Shared Memory. 19, 20, 23, 28–30, 34–39, 41–43, 46, 47, 49, 50, 58, 64

SO Shared Object. 8, 9, 11, 12, 27–29, 35–37, 41, 42, 45

TB Translation Block. 17

VMA Virtual Memory Area. 27

List of Figures

1.1 Generic diagram describing the Coverage-Guided Grey-box fuzzing workflow 3
1.2 An example of a Static Binary Rewriting Workflow 5
1.3 An example of Static Instrumentation using the trampoline technique . . . 6
1.4 An example of Lazy Binding involving .plt and .got 10
1.5 Dynamically linked ELF loading procedure 11
1.6 Example of a *.dynsym* table for a shared object 12
1.7 Example of a *.rela.dyn* relocation table extracted from a binary 12
1.8 Example of what happend during a function call 15
1.9 Example of what happend during a function call in case of frame pointer omission 15
1.10 Example of function call that does not allocate space of the stack bu access the variable saved on the red zone 16
1.11 Do-while CFG 20
1.12 AFL++ Forkserver Protocol 21

2.1 An example of ELF execution with dynamic dependencies and a constructor function 27
2.2 Shared Memory relocation between the target binary and the Shared Object 29
2.3 Example of the CFG of a program that computes the power operation between two numbers 33
2.4 Optimized forkserver invocation 38

3.1 Binary modification phase in which the needed sections, relocations and dynamic dependencies are added. (The dotted arrows specify which information are needed to following steps) 44
3.2 Example of the error caused by not taking care of RIP-relative instruction offsets 48
3.3 Example ax register during first instructions of the basic block instrumentation 52

List of Tables

2.1 RFLAGS register content and meaning 37

3.1 Different execution modes of the forkserver based on environment variable values 42

4.1 Binaries extracted from ZAFL *real-world software* dataset 56
4.2 Results of experiment I 57
4.3 Fuzzing statistics over a 24-hour fuzzing campaign on the LAVA-M dataset using AFL++ in QEMU mode, ZAFL instrumentation + AFL++ fuzzer, and my instrumentation + AFL++ fuzzer 59
4.4 LAVA-M uniq bugs triggered by the runs performed in table 4.3 60
4.5 Instrumented binaries statistics 61

List of Algorithms

1.1 Relocation resolution at ELF load time 13
1.2 ResolveSymbol function used by algorithm 1.1 14
1.3 AFL-style SHM update rule 20
1.4 Do-While Example 20
2.1 Instrumentation point search with DFS algorithm 31
2.2 Pseudo-code of Shared Memory update instrumentation 35
3.1 AFL forkserver algorithm 43
3.2 Initial Instrumentation algorithm 50
3.3 Instrument Basic Block algorithm 53